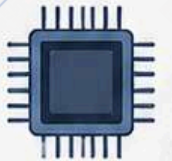


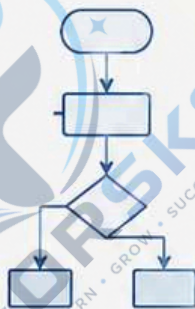
1010101010
0101010101
1010101010
0101010101



10110
01001
10101



Unit-I





DATA STRUCTURE

— INTRODUCTION TO DATA STRUCTURE —



A **Data Structure** is a way of **organizing** and **storing** data in a computer so that it can be used **efficiently**.

WHAT IS DATA STRUCTURE?

A data structure is a **collection** of data values, the **relationships** among them, and the **functions** or **operations** that can be applied to the data.

In simple words:

It is a way to **store**, **organize** and **manage** data in memory so that it can be accessed and modified efficiently.



WHY DO WE NEED DATA STRUCTURES?



Efficiency – Helps in processing data faster and using memory wisely.



Organization – Keeps data well structured and easy to manage.



Easy Access – Allows quick searching, insertion, deletion and other operations.



Problem Solving – Essential for solving complex real-world problems.

CHARACTERISTICS OF DATA STRUCTURE



Organization

Data is organized in a specific manner.



Storage

Data is stored in computer memory.



Operations

Supports various operations like insert, delete, search, update.



Efficiency

Designed to perform operations in the most efficient way possible.



Reusability

Data structures can be reused in different applications.

TYPES OF DATA STRUCTURES

Data structures are broadly classified into two types:

1. Linear Data Structure

Elements are arranged in a sequential manner.

Examples:

- Array
- Linked List
- Stack
- Queue



2. Non-Linear Data Structure

Elements are not arranged sequentially.

Examples:

- Tree
- Graph
- Heap



COMMON OPERATIONS



Insert – Add a new element into the data structure.



Delete – Remove an element from the data structure.



Search – Find the location of an element.



Update – Modify the value of an existing element.



Traverse – Access each element exactly once.



KEY TAKEAWAY

Data structures are the foundation of efficient programming.

Choosing the right data structure for the right problem makes programs faster, simpler and more effective.





TYPES OF DATA STRUCTURE



• Primitive & Non-Primitive •

Data structures are of **two main types** based on the type of data they store and the operations that can be performed.

1. PRIMITIVE DATA STRUCTURE

Primitive data structures are the basic data types that store a **single value**.

Examples

123

Integer
(int)

3.14

Float
(float)

'A'

Character
(char)

True

Boolean
(bool)

Features

- Store a single value.
- Built-in data types of a programming language.
- Directly supported by the system.
- Fixed size and simple.

Examples in C/Java

```
int a = 10;    float b = 3.5;    char ch = 'A';    boolean flag = true;
```

2. NON-PRIMITIVE DATA STRUCTURE

Non-primitive data structures are derived types that can store **multiple values**.

Examples

10 20 30 40

Array

10 → 20 → 30

Linked List



Stack

1 → 2 → 3 → 4

Queue



Tree



Graph



Hash Table

...

Others

Features

- Store multiple values (collection of data).
- User-defined or derived data types.
- More complex.
- Require more memory.
- Can represent relationships between data.

Examples in C/Java

```
int arr[4] = {10, 20, 30, 40};           // Array
struct Node { int data; struct Node* next; } // Linked List
Stack s;    Queue q;    Tree t;    Graph g;
```

PRIMITIVE vs NON-PRIMITIVE DATA STRUCTURE

Aspect	Primitive Data Structure	Non-Primitive Data Structure
Definition	Stores a single value.	Stores multiple values (collection of data).
Built-in / User-defined	Built-in (predefined).	User-defined.
Size	Fixed size.	Variable size.
Complexity	Simple.	Complex.
Memory	Less memory.	More memory.
Examples	int, float, char, bool	Array, Linked List, Stack, Queue, Tree, Graph, etc.
Supported Operations	Basic operations only.	Many operations (insert, delete, search, traverse, etc.).



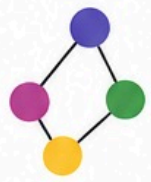
SUMMARY

Primitive data structures hold a single value and are simple. Non-primitive data structures hold multiple values and help in organizing complex data efficiently.





LINEAR AND NON-LINEAR DATA STRUCTURE



Data structures are broadly classified based on the **arrangement** of elements and the **relationships** between them.

1. LINEAR DATA STRUCTURE

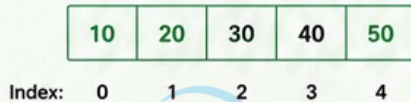
In linear data structures, elements are arranged in a **sequential order**. Each element (except the first and last) has exactly **one predecessor** and **one successor**.

Characteristics

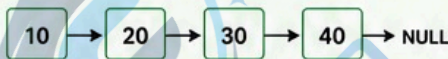
- Elements are stored in a sequence.
- There is a one-to-one relationship between elements.
- Traversal is done in a single path.

Examples

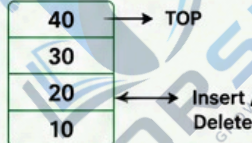
• Array



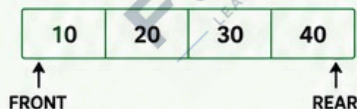
• Linked List



• Stack



• Queue



Applications



Managing lists of data



Function calls, undo operations



CPU scheduling, printer queue

2. NON-LINEAR DATA STRUCTURE

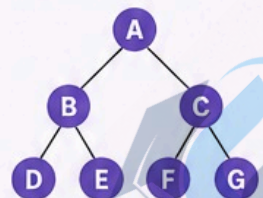
In non-linear data structures, elements are **not** arranged in a **sequential order**. One element can be connected to **multiple other elements**.

Characteristics

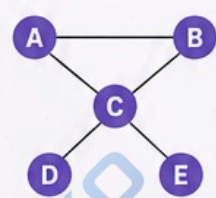
- Elements are not stored in sequence.
- There is a one-to-many or many-to-many relationship.
- Traversal may have multiple paths.

Examples

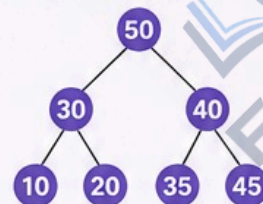
• Tree



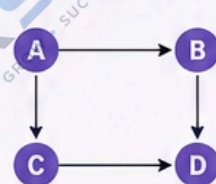
• Graph



• Heap



• Graph (Directed)



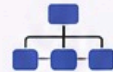
Applications



File systems, organization hierarchies



Network routing, social networks



Compilers, AI, databases

LINEAR vs NON-LINEAR DATA STRUCTURE

Aspect	Linear Data Structure	Non-Linear Data Structure
Arrangement	Elements are arranged in a sequence.	Elements are not arranged in sequence.
Relationship	One-to-one relationship.	One-to-many or many-to-many relationship.
Traversal	Single path.	Multiple paths.
Complexity	Generally simpler.	Generally more complex.
Memory Usage	Less memory (in most cases).	More memory (due to links/pointers).
Examples	Array, Linked List, Stack, Queue.	Tree, Graph, Heap, etc.
Applications	Simple applications, linear processing.	Complex applications, hierarchical relationships.



KEY TAKEAWAY

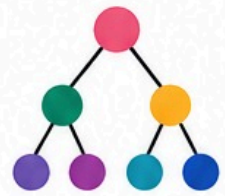
Use linear data structures when data is sequential.

Use non-linear data structures when data has hierarchical or network relationships.





DATA STRUCTURE OPERATIONS



Operations are the basic actions that can be performed on **data** structures to **manipulate** the **data** stored in them.

COMMON OPERATIONS

1
INSERT



Insertion is the process of **adding a new element** at a specific position in the data structure.

Example (Array)

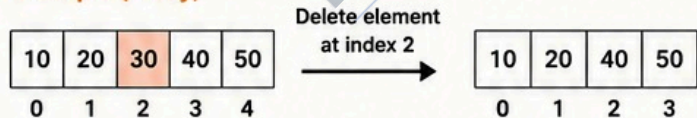


2
DELETE



Deletion is the process of **removing an element** from the data structure.

Example (Array)

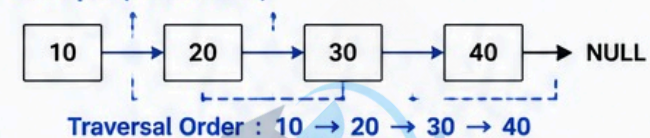


3
TRAVERSE



Traversal is the process of **visiting each element** of the data structure exactly once in a particular order.

Example (Linked List)

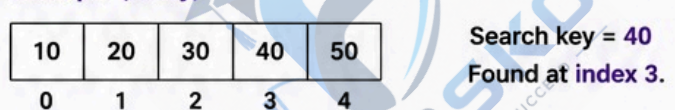


4
SEARCH



Searching is the process of **finding the location** (index or address) of an element in the data structure.

Example (Array)

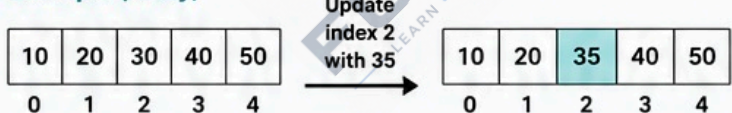


5
UPDATE



Updating is the process of **modifying the value** of an existing element.

Example (Array)

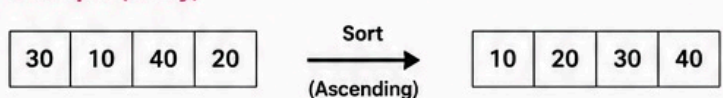


6
SORT



Sorting is the process of **arranging elements** in ascending or descending order.

Example (Array)



SUMMARY TABLE

Operation	Purpose	Where Used
1 Insert	Add a new element	Array, Linked List, Stack, Queue, Tree, Graph, etc.
2 Delete	Remove an existing element	All data structures
3 Traverse	Visit each element exactly once	Linked List, Tree, Graph, Array, etc.
4 Search	Find the position of an element	Array, Linked List, Tree, Graph, etc.
5 Update	Modify an existing element	Array, Linked List, Tree, Graph, etc.
6 Sort	Arrange elements in order	Array, List, etc.



KEY TAKEAWAY

These basic operations allow efficient manipulation of data in different data structures. The choice of data structure depends on the type of operations to be performed and the problem to be solved.





ARRAY



Definition and Concepts, Memory Representations



1. DEFINITION

An **array** is a collection of **similar data elements** stored in **contiguous memory locations** and accessed using a **common name**.

2. CONCEPTS

- All elements are of the same data type.
- Elements are stored in contiguous memory locations.
- Elements are accessed using an index (or subscript).
- The index of the first element is 0.
- For an array of size N, valid indices are 0 to N-1.
- Direct access is possible. Any element can be accessed in constant time $O(1)$.



3. BASIC TERMINOLOGIES

- **Element** : Each value stored in an array.
- **Index** : Position of an element in the array.
- **Size** : Total number of elements in the array.
- **Array Name** : Name used to represent the array.
- **Base Address** : Address of the first memory location of the array.
- **Contiguous** : Consecutive memory locations.

Example

Let A be an array of size 5.

$A = [10 \quad 20 \quad 30 \quad 40 \quad 50]$

Index: 0 1 2 3 4

Access:

$A[0] = 10$

$A[2] = 30$

$A[4] = 50$

4. MEMORY REPRESENTATIONS OF ARRAY

4.1 ONE-DIMENSIONAL ARRAY (1D)

An array of elements arranged in a single row.

Example: $A[5] = \{10, 20, 30, 40, 50\}$

Array Elements	→	10	20	30	40	50
Index	→	0	1	2	3	4
Memory Address	→	1000	1004	1008	1012	1016

- Assume base address (BA) = 1000
- If each integer occupies 4 bytes.

4.2 TWO-DIMENSIONAL ARRAY (2D)

An array of elements arranged in rows and columns (matrix form).

Example: $A[3][4]$

		Column Index (j)			
		0	1	2	3
Row Index (i)	0	10	20	30	40
	1	50	60	70	80
	2	90	100	110	120

Access:

$A[0][0] = 10$

$A[1][2] = 70$

$A[2][3] = 120$

- Stored in memory in **row-major order** (row by row).

4.3 ADDRESS CALCULATION

Let,

BA = Base Address of array

W = Size of each element in bytes

i, j = Row and Column indices

N = Number of columns

(a) 1D Array

Address of $A[i]$

$$= BA + i \times W$$

Where, $0 \leq i < N$

(b) 2D Array (Row-Major Order)

Address of $A[i][j]$

$$= BA + ((i \times N) + j) \times W$$

Where, $0 \leq i < \text{Rows}$, $0 \leq j < \text{Columns}$

Note:

Row-major order stores row by row in contiguous memory locations.

5. EXAMPLE: ADDRESS CALCULATION

(a) 1D Array

$A[5]$, BA = 1000, W = 4 bytes

Find address of $A[3]$

$$\text{Address} = 1000 + 3 \times 4 = 1012$$

(b) 2D Array

$A[3][4]$, BA = 2000, W = 4 bytes, N = 4 (columns)

Find address of $A[2][3]$

$$\begin{aligned} \text{Address} &= 2000 + ((2 \times 4) + 3) \times 4 \\ &= 2000 + (11 \times 4) = 2044 \end{aligned}$$

6. CHARACTERISTICS OF ARRAY

- ✓ Fixed size (static memory allocation).
- ✓ Homogeneous elements (same data type).
- ✓ Contiguous memory allocation.
- ✓ Random access: $O(1)$ time.
- ✓ Efficient traversal using loops.
- ✓ Insertion and deletion are costly (because of shifting).



7. COMMON OPERATIONS

- Traversal (visit each element)
- Insertion (at specific position)
- Deletion (at specific position)
- Searching (linear / binary)
- Sorting (ascending / descending)
- Updating (modify element)



KEY TAKEAWAY

Arrays store similar elements in contiguous memory locations and provide direct access to any element using its index. Understanding their memory representation is essential for efficient programming.





ARRAY OPERATIONS



Traversing, Insertion, Deletion

An array is a collection of **similar elements** stored in **contiguous memory** locations and accessed using an **index**.

1. TRAVERSING AN ARRAY

Traversing means **visiting each element** of the array exactly once in a specific order (usually from first to last).

Example

Let A =

10	20	30	40	50
----	----	----	----	----

Index:

0	1	2	3	4
---	---	---	---	---



Traversal Order: 10 → 20 → 30 → 40 → 50



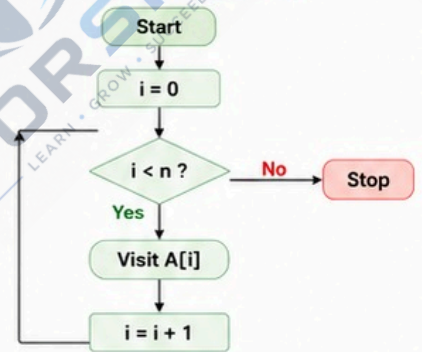
ALGORITHM (TRAVERSAL)

```
for i = 0 to n-1
    visit A[i]
end for
```

PSEUDOCODE

```
for i ← 0 to n - 1
    print A[i]
end for
```

Flow / Illustration



2. INSERTION IN AN ARRAY

Insertion means **adding a new element** at a specific position. Elements from that position are shifted one place to the right.

Example

Insert 25 at index 2 in A =

10	20	30	40	50
----	----	----	----	----

 (Size = 5)

Step 1: Shift elements to the right from index 2

Index:

0	1	2	3	4
10	20	30	40	50

Step 2: Insert 25 at index 2

Index:

0	1	2	3	4	5
10	20	25	30	40	50



ALGORITHM (INSERTION)

1. Check if the array is full.
2. For $i = n - 1$ down to pos
 $A[i + 1] = A[i]$
3. $A[\text{pos}] = \text{newElement}$
4. $n = n + 1$

PSEUDOCODE

```
if n == MAX
    print "Array Overflow"
else
    for i ← n-1 downto pos
        A[i+1] ← A[i]
    A[pos] ← newElement
    n ← n + 1
end if
```

Illustration

Before Insertion

Index:

0	1	2	3	4
10	20	30	40	50

Insert 25 at index 2

After Insertion

Index:

0	1	2	3	4	5
10	20	25	30	40	50

3. DELETION FROM AN ARRAY

Deletion means **removing an element** from a specific position. Elements after that position are shifted one place to the left.

Example

Delete element at index 2 (value 30) from A =

10	20	30	40	50
----	----	----	----	----

 (Size = 5)

Step 1: Shift elements to the left from index 2

Index:

0	1	2	3	4
10	20	30	40	50

Step 2: Reduce size by 1

Index:

0	1	2	3
10	20	40	50



ALGORITHM (DELETION)

1. Check if the array is empty.
2. For $i = \text{pos}$ to $n - 2$
 $A[i] = A[i + 1]$
3. $n = n - 1$

PSEUDOCODE

```
if n == 0
    print "Array Underflow"
else
    for i ← pos to n-2
        A[i] ← A[i+1]
    n ← n - 1
end if
```

Illustration

Before Deletion

Index:

0	1	2	3	4
10	20	30	40	50

Delete element at index 2 (30)

After Deletion

Index:

0	1	2	3
10	20	40	50

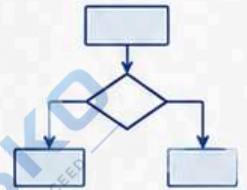


KEY TAKEAWAY

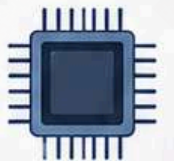
- Traversing visits each element of the array.
- Insertion adds a new element and may require shifting.
- Deletion removes an element and may require shifting.



1010101010
0101010101
1010101010
0101010101



10110
01001
10101



Unit - II



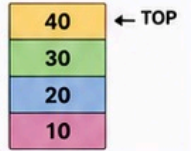


STACKS

Definition and Concepts, Memory Representations

LIFO Principle

Stack follows LIFO (Last In First Out) principle.

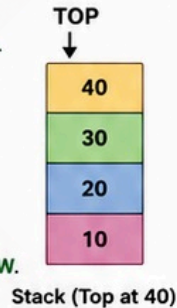


1. DEFINITION

A stack is a **linear** data structure that follows the **LIFO (Last In First Out)** principle. In a stack, insertion and deletion of elements can be performed from only **one end** called **TOP**.

2. CONCEPTS

- Stack is a collection of **similar data elements**.
- All operations (insertions and deletions) are performed at one end called **TOP**.
- The element at the top is the most recently inserted element.
- When a stack is full, it is called **OVERFLOW**.
- When a stack is empty, it is called **UNDERFLOW**.

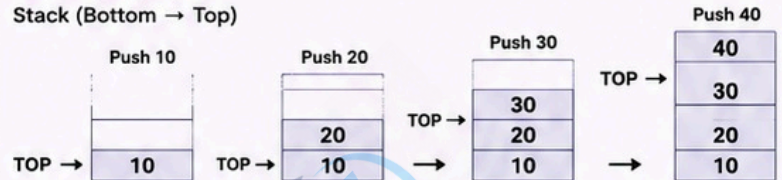


3. BASIC OPERATIONS

Operation	Description	Time Complexity
Push	Insert an element at the TOP	O(1)
Pop	Delete the element from the TOP	O(1)
Peek/Top	Return the top element	O(1)
isEmpty	Check if stack is empty	O(1)
isFull	Check if stack is full	O(1)

Example

Push 10, 20, 30, 40
Stack (Bottom → Top)



4. MEMORY REPRESENTATIONS OF STACK

4.1 STACK USING ARRAY

The stack is implemented using a one-dimensional array.

Representation

Index	Value	TOP
4	50	← TOP
3	40	
2	30	
1	20	
0	10	

(Bottom)

Example

After pushing 10, 20, 30, 40, 50
TOP = 4 (points to 50)
Elements are stored in contiguous memory.

Notes

- MAX = 5 (size of stack)
- TOP = -1 → Stack is empty
- When TOP = MAX - 1 → Stack is full (**Overflow**)
- New element is inserted at ++TOP
- Element is deleted from TOP--

4.2 STACK MEMORY LAYOUT

Memory addresses grow from lower to higher.
Stack grows **upwards** in memory.

Memory Address	Value	TOP
1008	50	← TOP (points to 1008)
1004	40	
1000	30	
996	20	
992	10	
...	...	
Lower Addresses		

Growth Direction (Upwards)

Example

Base address (BA) = 992
Size of each element = 4 bytes
MAX = 5
Then,
Address of element at index i
= BA + (i × size)
For index 3 (40):
= 992 + (3 × 4)
= 1004

5. UNDERFLOW AND OVERFLOW

Underflow

Occurs when POP or PEEK is performed on an empty stack (TOP = -1).



Overflow

Occurs when PUSH is performed on a full stack (TOP = MAX - 1).



6. APPLICATIONS OF STACK

- Function calls (Call Stack)
- Recursion
- Expression evaluation and conversion
- Backtracking (e.g., maze, N-Queens)
- Syntax checking (e.g., parentheses matching)
- Undo operations in editors / browsers



7. SUMMARY TABLE

Feature	Stack
Type	Linear Data Structure
Principle	LIFO (Last In First Out)
Insertion	At TOP only (Push)
Deletion	From TOP only (Pop)
Access	Only TOP element accessible
Memory	Contiguous (Array) / Dynamic (Linked List)
Time Complexity	O(1) for all basic operations



KEY TAKEAWAY

Stacks follow LIFO principle. All operations are performed at TOP. Using arrays, stack elements are stored in contiguous memory locations and grow upwards.





STACK OPERATIONS

Traversing, Insertion, Deletion

Stack (Top at right)



Stack follows **LIFO** (Last In First Out) principle.
All operations are performed at one end called **TOP**.

1 TRAVERSING A STACK

Traversing means accessing/visiting each element of the stack from **TOP** to **BOTTOM** without removing any element.

Example

Stack elements (Bottom → Top):

10	20	30	40
----	----	----	----

Indices: 0 1 2 3

TOP



BOTTOM

ALGORITHM (TRAVERSAL)

1. If stack is empty, return.
2. Set a temporary variable temp = TOP.
3. While temp ≠ -1
 - Print/visit STACK[temp]
 - temp = temp - 1
4. Stop.

PSEUDOCODE

```
if TOP == -1
    print "Stack is Empty"
else
    temp = TOP
    while temp != -1
        print STACK[temp]
        temp = temp - 1
```

Illustration (Traversal)

Step	temp	Element Visited	Order
Initial	3	—	—
1	3	40	40
2	2	30	40, 30
3	1	20	40, 30, 20
4	0	10	40, 30, 20, 10
5	-1	Stop	Done

Traversal Order: 40 → 30 → 20 → 10
(From TOP to BOTTOM)

2 INSERTION (PUSH) IN STACK

Insertion operation is called **PUSH**.
A new element is inserted at TOP.

Example

Initial Stack (Bottom → Top):

10	20	30
----	----	----

TOP = 2

Push 40 into stack.

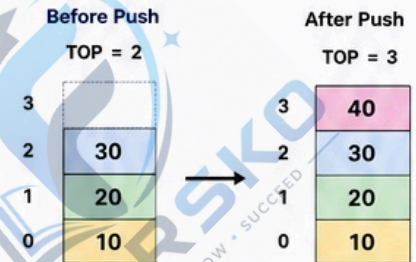
ALGORITHM (PUSH)

1. If TOP == MAX - 1
→ Stack Overflow
2. Else
→ TOP = TOP + 1
→ STACK[TOP] = newElement

PSEUDOCODE

```
if TOP == MAX - 1
    print "Stack Overflow"
else
    TOP = TOP + 1
    STACK[TOP] = newElement
```

Illustration (Push 40)



TOP is updated to 3 and 40 is inserted.
(Stack size increased by 1)

3 DELETION (POP) FROM STACK

Deletion operation is called **POP**.
The element at TOP is removed.

Example

Initial Stack (Bottom → Top):

10	20	30	40
----	----	----	----

TOP = 3

Pop the top element.

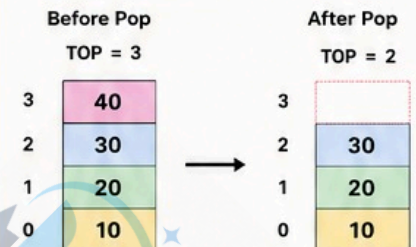
ALGORITHM (POP)

1. If TOP == -1
→ Stack Underflow
2. Else
→ element = STACK[TOP]
→ TOP = TOP - 1
→ Return element

PSEUDOCODE

```
if TOP == -1
    print "Stack Underflow"
else
    element = STACK[TOP]
    TOP = TOP - 1
    return element
```

Illustration (Pop)



Popped element = 40
TOP is updated to 2.
(Stack size decreased by 1)

SUMMARY TABLE

Operation	Purpose	Effect on TOP	Time Complexity
Traversal	Visit each element from TOP to BOTTOM	No Change	O(n)
Insertion (Push)	Add new element at TOP	TOP = TOP + 1	O(1)
Deletion (Pop)	Remove element from TOP	TOP = TOP - 1	O(1)

LEGEND

MAX = Maximum size of stack
TOP = Index of top element
-1 = Stack is empty

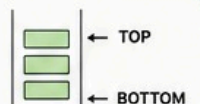
Push(x) : Insert x at TOP

Pop() : Remove and return top element

Peek/Top() : Return top element without removing it

isEmpty() : Check if stack is empty

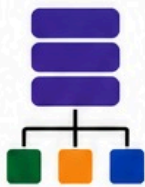
isFull() : Check if stack is full



KEY TAKEAWAY

- ✓ Stack follows LIFO (Last In First Out) principle.
- ✓ All operations (except traversal) are done at TOP only.
- ✓ Overflow: Occurs when trying to push in a full stack.
- ✓ Underflow: Occurs when trying to pop from an empty stack.
- ✓ Traversal does not remove elements; it only visits them.



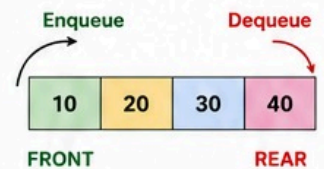


QUEUE

Definition and Concepts, Memory Representations

FIFO Principle

Queue follows FIFO (First In First Out) principle.



1. DEFINITION

A queue is a linear data structure that follows **FIFO (First In First Out)** principle.

Insertion of an element is done at one end called **REAR** and deletion is done from the other end called **FRONT**.

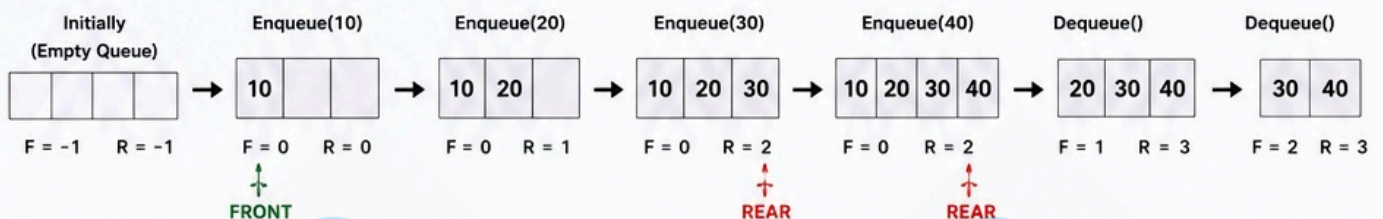
2. CONCEPTS

- Elements are added at **REAR** (Enqueue).
- Elements are removed from **FRONT** (Dequeue).
- The first element inserted is the first element removed.
- If queue is full, insertion causes **OVERFLOW**.
- If queue is empty, deletion causes **UNDERFLOW**.

3. BASIC OPERATIONS

Operation	Description	Time Complexity
Enqueue	Insert an element at REAR	O(1)
Dequeue	Delete an element from FRONT	O(1)
Front/PEEK	Return the front element	O(1)
Rear	Return the rear element	O(1)
isEmpty	Check if queue is empty	O(1)
isFull	Check if queue is full	O(1)

Example Perform Enqueue(10), Enqueue(20), Enqueue(30), Enqueue(40), Dequeue(), Dequeue().

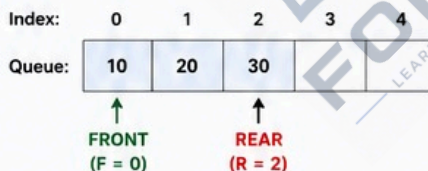


4. MEMORY REPRESENTATIONS OF QUEUE

4.1 QUEUE USING ARRAY (LINEAR QUEUE)

Queue is implemented using a one-dimensional array. Elements are stored in contiguous memory locations.

Representation



Notes

- MAX = 5 (size of array)
- F = Front index
- R = Rear index
- Initially: F = -1, R = -1
- Queue is Full when R = MAX - 1
- Queue is Empty when F = -1 or F > R

Example

Array size = 5

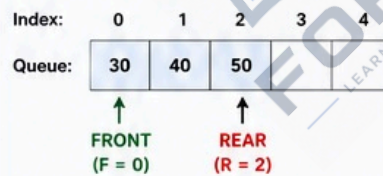
Insert 10, 20, 30, 40



4.2 QUEUE USING CIRCULAR ARRAY (CIRCULAR QUEUE)

To efficiently utilize memory, the last position is connected to the first position to form a circle.

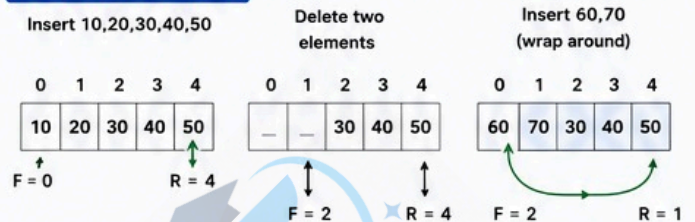
Representation



Notes

- MAX = 5
- Next position after last index (4) is 0
- Condition for Full: $(R + 1) \% MAX = F$
- Condition for Empty: F = -1

Example (Circular Behavior)



5. UNDERFLOW AND OVERFLOW

Underflow

Occurs when DEQUEUE or FRONT/PEEK is performed on an empty queue. (No element to remove).



Overflow

Occurs when ENQUEUE is performed on a full queue. (No space to insert).



6. APPLICATIONS OF QUEUE

- ✓ CPU scheduling (Round Robin)
- ✓ Process management in OS
- ✓ Printer spooling
- ✓ I/O buffering
- ✓ Breadth First Search (BFS) in graphs
- ✓ Real-world ticket counters, customer lines, etc.

TICKET COUNTER



7. SUMMARY TABLE

Feature	Linear Queue	Circular Queue
Structure	Array (Linear)	Array (Circular)
Space Utilization	May waste space (after deletions)	Better space utilization
Full Condition	$R = MAX - 1$	$(R + 1) \% MAX = F$
Empty Condition	$F = -1$ or $F > R$	$F = -1$
Complexity	O(1) for all basic operations	O(1) for all basic operations



KEY TAKEAWAY

- Queue follows FIFO (First In First Out) principle.
- Insertion is done at REAR and deletion is done from FRONT.
- Circular queue is preferred to avoid memory wastage.

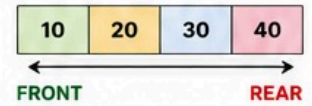




QUEUE OPERATIONS:

TRAVERSING, INSERTION, DELETION & TYPES OF QUEUE

Queue follows **FIFO**
(First In First Out)



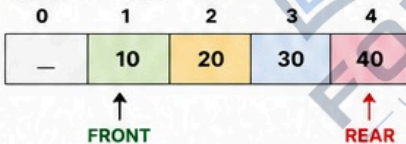
A queue is a linear data structure that follows **FIFO** (First In First Out) principle.
Insertion is done at **REAR** and deletion is done from **FRONT**.

1 TRAVERSING (DISPLAY)

Traversing means visiting/printing all elements of the queue from **FRONT** to **REAR** without removing any element.

Example

Queue (F = 1, R = 4)



Algorithm

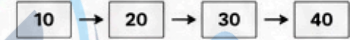
1. If queue is empty, return.
2. Set $i = \text{FRONT}$.
3. Repeat until $i = \text{REAR}$
 - (a) Visit/print $Q[i]$
 - (b) $i = i + 1$
4. Stop.

Pseudocode

```
if (F == -1)
    print "Queue is Empty"
else
    for i = F to R
        print Q[i]
```

Illustration (Traversal)

Output:



Step	i	Element Visited	Order
Initial	1	-	-
1	1	10	10
2	2	20	10, 20
3	3	30	10, 20, 30
4	4	40	10, 20, 30, 40
End	-	Done	Done

Traversal Order: 10 → 20 → 30 → 40
(From **FRONT** to **REAR**)

2 INSERTION (ENQUEUE)

Insertion operation is called **ENQUEUE**.
A new element is inserted at **REAR**.

Example

Initial Queue (F = 1, R = 3)



Enqueue(40)

After Insertion (F = 1, R = 4)



Algorithm (Enqueue)

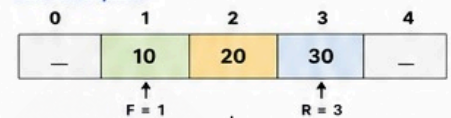
1. If queue is full, report **OVERFLOW**.
2. If queue is empty
Set $F = R = 0$
3. Else
Set $R = R + 1$
4. Insert new element at $Q[R]$.

Pseudocode

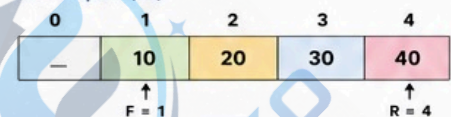
```
if (queue is full)
    print "Queue Overflow"
else if (queue is empty)
    F = R = 0
else
    R = R + 1
    Q[R] = newElement
```

Illustration (Enqueue 40)

Before Enqueue



After Enqueue(40)



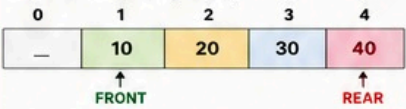
REAR is updated and 40 is inserted.
(Queue size increases by 1)

3 DELETION (DEQUEUE)

Deletion operation is called **DEQUEUE**.
An element is removed from **FRONT**.

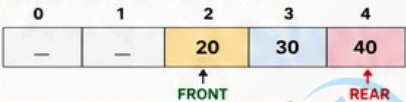
Example

Initial Queue (F = 1, R = 4)



Dequeue()

After Deletion (F = 2, R = 4)



Algorithm (Dequeue)

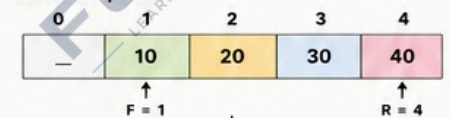
1. If queue is empty, report **UNDERFLOW**.
2. Store and remove element at **FRONT**.
3. If $F = R$ (only one element was present)
Set $F = R = -1$ (Queue becomes empty)
4. Else
Set $F = F + 1$

Pseudocode

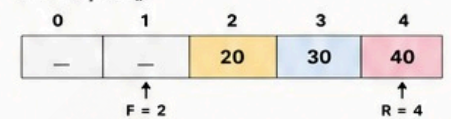
```
if (queue is empty)
    print "Queue Underflow"
else
    removedElement = Q[F]
    if (F == R)
        F = R = -1 // Queue becomes empty
    else
        F = F + 1
    return removedElement
```

Illustration (Dequeue)

Before Dequeue



After Dequeue()

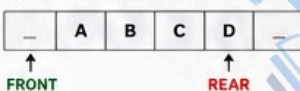


Element 10 is removed.
FRONT is updated. (Queue size decreases by 1)

4 TYPES OF QUEUE

1. LINEAR QUEUE

Insertion at **REAR** and deletion from **FRONT** in a linear manner.



- Simple implementation
- May cause wastage of space (insertion limited by size)

2. CIRCULAR QUEUE

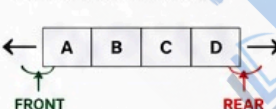
Last position is connected to the first position to form a circle.



- Efficient memory utilization
- No space wastage

3. DEQUE (Double Ended Queue)

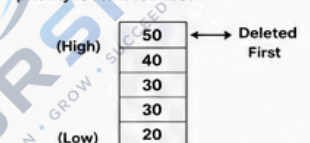
Insertion and deletion are allowed at both **FRONT** and **REAR**.



- More flexible
- Can act as queue or stack

4. PRIORITY QUEUE

Each element has a priority. Element with highest (or lowest) priority is removed first.



- Based on priority
- Used in scheduling, etc.

SUMMARY TABLE (OPERATIONS)

Operation	Purpose	Condition Checked	Time Complexity
Traversal (Display)	Visit all elements from FRONT to REAR	Check if Empty	$O(n)$
Insertion (Enqueue)	Add element at REAR	Check if Full	$O(1)$
Deletion (Dequeue)	Remove element from FRONT	Check if Empty	$O(1)$

CONDITIONS

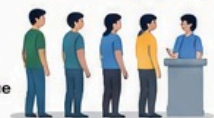
- Queue is Empty → $F = -1$ and $R = -1$
- Queue is Full (Linear Queue) → $R = \text{MAX} - 1$
- Queue is Full (Circular Queue) → $(R + 1) \% \text{MAX} = F$
- Queue is Empty (Circular Queue) → $F = -1$



KEY TAKEAWAY

- ✓ Queue follows **FIFO** principle.
- ✓ Insertion is done at **REAR**.
- ✓ Deletion is done from **FRONT**.
- ✓ Choose the right type of queue as per the application.

TICKET COUNTER



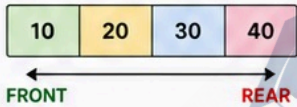
TYPES OF QUEUE

Queue is a linear data structure that follows **FIFO (First In First Out)** principle.

Common Terms

- FRONT** : Position of first element
REAR : Position of last element
ENQUEUE : Insert at REAR
DEQUEUE : Delete from FRONT

FIFO Principle

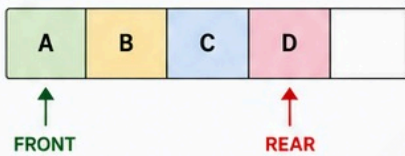


Insertion at **REAR**
 Deletion from **FRONT**

1. LINEAR QUEUE

Elements are inserted at **REAR** and deleted from **FRONT** in a linear manner.

Diagram



Features

- Simple structure
- Easy to implement
- May cause wastage of space (empty positions cannot be used)

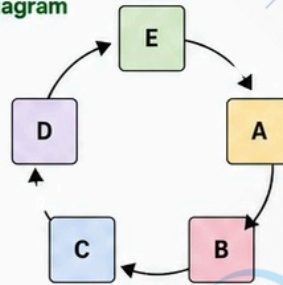
Applications

- CPU scheduling
- Print spooling
- Basic buffering

2. CIRCULAR QUEUE

Last position is connected to the first position to form a circle.

Diagram



Features

- Efficient memory utilization
- No space wastage
- Used modulo arithmetic for indexing

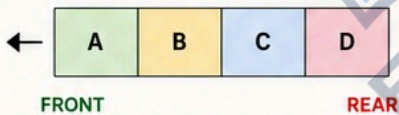
Applications

- Real-time systems
- I/O buffering
- Circular scheduling

3. DEQUE (DOUBLE ENDED QUEUE)

Insertion and deletion are allowed at both **FRONT** and **REAR**.

Diagram



Operations

- Insert at FRONT
- Insert at REAR
- Delete from FRONT
- Delete from REAR

Features

- More flexible than simple queue
- Can be implemented using array or linked list

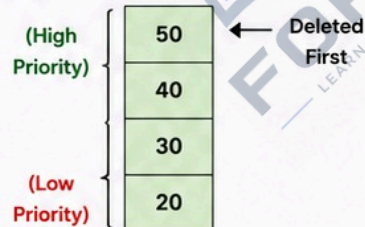
Applications

- Palindrome checking
- Undo/Redo operations
- Sliding window problems

4. PRIORITY QUEUE

Each element has a priority. Element with highest (or lowest) priority is removed first.

Diagram



Features

- Elements are served based on priority
- Higher (or lower) priority element is dequeued first
- Can have same priority for multiple elements

Applications

- Job scheduling
- Dijkstra's algorithm
- Emergency services
- Process management

COMPARISON OF QUEUE TYPES

Type	Insertion	Deletion	Ends Used	Memory Utilization	Best Used For
Linear Queue	At REAR	From FRONT	One (Linear)	May waste space	Simple applications
Circular Queue	At REAR (modulo)	From FRONT (modulo)	One (Circular)	No space waste	When memory is limited
Deque	At both ends	From both ends	Two (Both ends)	No space waste (if circular)	Undo/Redo, Palindrome, Sliding window
Priority Queue	By Priority	By Priority	One	Depends on implementation	Scheduling, Path finding, Priority based tasks

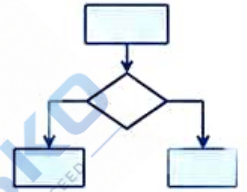


KEY TAKEAWAY

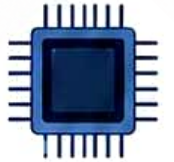
- ✓ All queues follow FIFO principle.
- ✓ Linear Queue is simplest but may waste space.
- ✓ Circular Queue is efficient in memory utilization.
- ✓ Deque allows operations at both ends.
- ✓ Priority Queue serves by priority, not by arrival order.



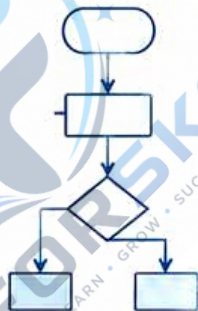
1010101010
0101010101
1010101010
0101010101

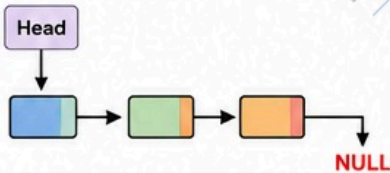


10110
01001
10101



Unit - III





LINKED LIST

Definition and Concepts, Memory Representations

What is Linked List?

A linked list is a linear data structure in which elements (nodes) are not stored in contiguous memory locations. Each node stores data and a reference (address) to the next node in the sequence.

1. DEFINITION

A linked list is a dynamic data structure consisting of a collection of **nodes**. Each node contains two parts:

- **DATA:** Stores the actual value
- **LINK (REFERENCE):** Stores the address of the next node in the list

Unlike arrays, elements in a linked list are not stored in contiguous memory locations.

2. CONCEPTS

- **Dynamic Size:** Can grow or shrink at runtime.
- **Non-contiguous Memory Allocation:** Nodes are scattered in memory.
- **Efficient Insertions/Deletions:** No need to shift elements; only links (addresses) are updated.
- **Access:** Sequential access only (no random access like arrays).
- **Requires Extra Memory:** Additional memory is needed to store addresses.

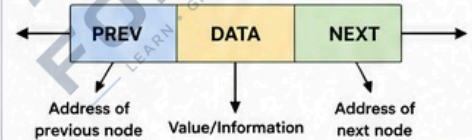
3. NODE STRUCTURE

A node is the basic building block.

Singly Linked List Node



Doubly Linked List Node

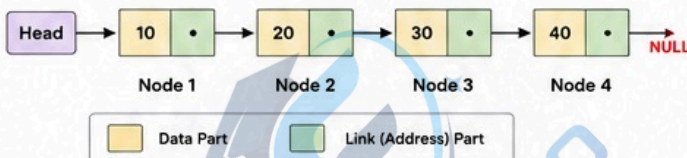


4. MEMORY REPRESENTATIONS

4.1 SINGLY LINKED LIST

Each node points to the next node. The last node's link part contains NULL.

Memory Representation Diagram



Example in Memory

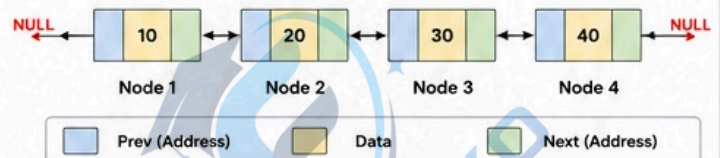
Node	Address	Data	Link (Address of Next Node)
Node 1	1000	10	2000
Node 2	2000	20	3000
Node 3	3000	30	4000
Node 4	4000	40	NULL

Nodes are stored at different memory locations.

4.2 DOUBLY LINKED LIST

Each node has two links: one to the next node and one to the previous node.

Memory Representation Diagram



Example in Memory

Node	Address	Prev (Address)	Data	Next (Address)
Node 1	1000	NULL	10	2000
Node 2	2000	1000	20	3000
Node 3	3000	2000	30	4000
Node 4	4000	3000	40	NULL

Traversal can be done in both forward (Next) and backward (Prev) directions.

5. ADDRESS CALCULATION (CONCEPTUAL)

In Linked List:

- Nodes are allocated dynamically using memory allocation functions (e.g., malloc() in C).
- Memory addresses are not consecutive.
- Links store addresses (references) of next/prev nodes.

Example:

- If address of Node 1 = 1000
- Node 1 points to Node 2 whose address = 2000
- Node 2 points to Node 3 whose address = 3000
- And so on...

Advantage of Linked List Memory Representation

- ✓ Efficient memory utilization
- ✓ Easy insertions and deletions
- ✓ Dynamic size management

6. TYPES OF LINKED LIST

1. SINGLY LINKED LIST

Each node contains data and address of next node.

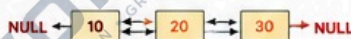


Features

- Traversal in one direction only
- Less memory (one link field)
- Simple and efficient

2. DOUBLY LINKED LIST

Each node contains address of previous node and next node.



Features

- Traversal in both directions
- More memory (two link fields)
- Easy deletion of a given node

3. CIRCULAR LINKED LIST

Last node points back to the first node.



Features

- No NULL at the end
- Useful in cyclic data processing
- Can start traversal from any node

4. DOUBLY CIRCULAR LINKED LIST

Doubly linked list in which last node connects to the first node and vice versa.



Features

- Bidirectional circular traversal
- No NULL at either end
- Useful in advanced applications



KEY TAKEAWAY

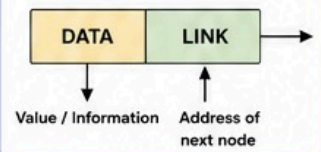
Linked List is a dynamic and flexible data structure where elements are connected using links (addresses). It allows efficient insertions and deletions, but sacrifices random access and requires extra memory for links.



TYPES OF LINKED LIST

A linked list is a linear data structure in which elements (nodes) are not stored in contiguous memory locations. Each node contains data and link(s) to other node(s).

Node Structure



MAIN TYPES OF LINKED LIST

1. SINGLY LINKED LIST

Each node contains data and address of next node. The last node points to NULL.

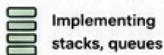
Structure Diagram



Features

- Traversal is possible in one direction only (forward).
- Less memory (one link field per node).
- Simple to implement.
- Efficient for insertions and deletions.

Applications



Implementing stacks, queues

$$a_n x^n + \dots + a_0$$

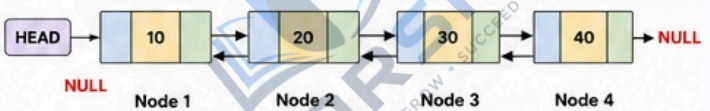


Memory management

2. DOUBLY LINKED LIST

Each node contains data and addresses of both next node and previous node.

Structure Diagram



Features

- Traversal possible in both forward and backward directions.
- More memory (two link fields per node).
- Easy deletion of a given node.
- Slower than singly linked list due to extra links.

Applications



Browser back/forward functionality



Music playlists

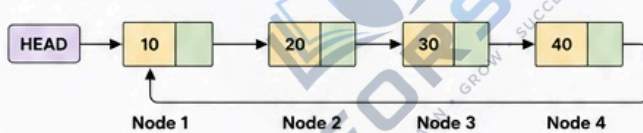


Undo / Redo operations

3. CIRCULAR LINKED LIST (SINGLY)

Last node points back to the first node. There is no NULL at the end.

Structure Diagram



Features

- Traversal can start from any node and continue in a circle.
- No NULL at the end.
- Efficient for circular data processing.
- Less memory (one link field per node).

Applications



Round Robin scheduling



Circular queues

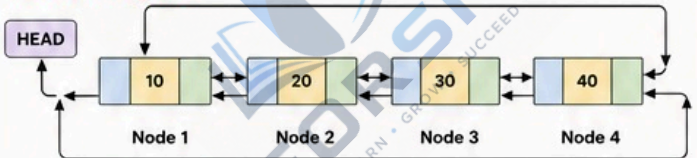


Multiplayer games

4. DOUBLY CIRCULAR LINKED LIST

Each node has two links (prev and next) and last node connects to first node and first node connects to last node.

Structure Diagram



Features

- Traversal in both directions in a circular manner.
- No NULL at both ends.
- More memory (two link fields per node).
- Most flexible among linked lists.

Applications



Circular deques



Advanced scheduling systems



Operating system process lists

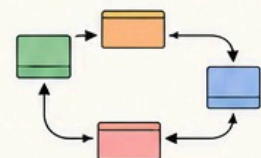
COMPARISON OF LINKED LIST TYPES

Type	Links per Node	Direction of Traversal	NULL at End	Memory Usage	Advantages	Disadvantages	Best Used For
Singly Linked List	1 (Next)	One (Forward)	Yes	Low	Simple, less memory, easy insertion	One-way traversal, hard to delete a given node	Stacks, Queues, Simple applications
Doubly Linked List	2 (Prev & Next)	Both	Yes	Medium	Two-way traversal, easy deletion	More memory, slightly complex	Browser history, Undo/Redo, Playlists
Circular Linked List (Singly)	1 (Next)	One (Circular)	No	Low	No NULL, can start from any node	One-way only, hard to delete	Round robin, Circular queues
Doubly Circular Linked List	2 (Prev & Next)	Both (Circular)	No	High	Most flexible, two-way circular	More memory, complex implementation	Dequeues, Advanced scheduling

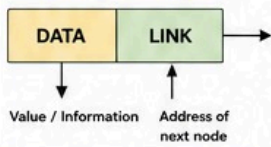


KEY TAKEAWAY

- ★ Linked lists are dynamic structures made of nodes connected by links.
- ★ Singly Linked List is simple and memory efficient but one-way.
- ★ Doubly Linked List allows two-way traversal and easy deletion.
- ★ Circular Linked List is useful for circular data processing and scheduling.
- ★ Doubly Circular Linked List is the most flexible but uses more memory.



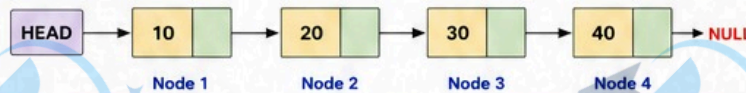
Node Structure



LINKED LIST OPERATIONS

TRAVERSING, INSERTION, DELETION

In a Singly Linked List, each node points to the next node and the last node points to NULL.



1. TRAVERSING (DISPLAY)

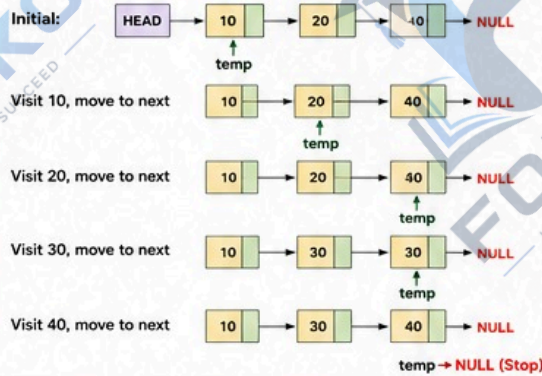
Traversing means visiting each node of the linked list from the beginning (HEAD) to the end (NULL) and processing (e.g., printing) the data.

Algorithm

1. Start from HEAD.
2. Visit (process) the data part of the current node.
3. Move to the next node using the link part.
4. Repeat steps 2 and 3 until the next pointer is NULL.

```
Pseudocode
temp = HEAD
while (temp != NULL)
{
    visit(temp->data)
    temp = temp->link
}
```

Step-by-Step Example



Key Points

- Traversal is one-way (from HEAD to NULL).
- Time Complexity: $O(n)$
- Space Complexity: $O(1)$
- Used to display or search elements.

Example Output

10 20 30 40

2. INSERTION

Insertion means adding a new node at a specified position.

Types of Insertion

1. At Beginning
2. At End
3. At Given Position (after a given node)

2.1 INSERTION AT BEGINNING

Steps

1. Create a new node.
2. Set new node's link = HEAD.
3. Update HEAD = new node.

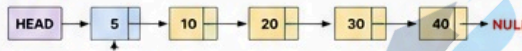
```
Pseudocode
new_node = (data)
new_node->link = HEAD
HEAD = new_node
```

Example: Insert 5 at the beginning

Before Insertion



After Insertion



2.2 INSERTION AT END

Steps

1. Create a new node.
2. If list is empty, set HEAD = new node.
3. Else, move to last node.
4. Set last node's link = new node.

```
Pseudocode
new_node = (data)
new_node->link = NULL
if (HEAD == NULL)
    HEAD = new_node
else
    temp = HEAD
    while (temp->link != NULL)
        temp = temp->link
    temp->link = new_node
```

2.3 INSERTION AFTER A GIVEN NODE (KEY)

Steps

1. Search for the node with data = KEY.
2. Create a new node.
3. Set new node's link = (node after KEY).
4. Set KEY node's link = new node.

```
Pseudocode
find node with data = KEY
if (node found)
{
    new_node = (data)
    new_node->link = key_node->link
    key_node->link = new_node
}
```

Key Points

- Insertion is easy and efficient in linked list.
- No need to shift elements (unlike arrays).
- Time Complexity:
 - At Beginning: $O(1)$
 - At End: $O(n)$
 - At Given Position: $O(n)$
- Space Complexity: $O(1)$

3. DELETION

Deletion means removing a node from the linked list.

Types of Deletion

1. From Beginning
2. From End
3. From Given Position (after a given node or by key)

3.1 DELETION FROM BEGINNING

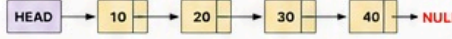
Steps

1. If list is empty, Underflow.
2. Store HEAD in temp.
3. Move HEAD to next node.
4. Delete temp.

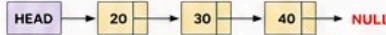
```
Pseudocode
if (HEAD == NULL)
    Underflow
else
{
    temp = HEAD
    HEAD = HEAD->link
    free(temp)
}
```

Example: Delete 10 (first node)

Before Deletion



After Deletion



3.2 DELETION FROM END

Steps

1. If list empty or one node, handle separately.
2. Move to second last node.
3. Delete last node.

```
Pseudocode
if (HEAD == NULL) Underflow
else if (HEAD->link == NULL)
{
    free(HEAD)
    HEAD = NULL
}
else
{
    temp = HEAD
    while (temp->link->link != NULL)
        temp = temp->link
    free(temp->link)
    temp->link = NULL
}
```

3.3 DELETION OF A GIVEN NODE (KEY)

Steps

1. If first node is key, delete it.
2. Else, find node just before key node.
3. Change link to skip the key node.
4. Delete the key node.

```
Pseudocode
if (HEAD->data == KEY)
{
    temp = HEAD
    HEAD = HEAD->link
    free(temp)
}
else
{
    prev = NULL
    curr = HEAD
    while (curr != NULL and curr->data != KEY)
    {
        prev = curr
        curr = curr->link
    }
    if (curr == NULL) Not Found
    else
    {
        prev->link = curr->link
        free(curr)
    }
}
```

Key Points

- Deletion requires finding the previous node (except for first node).
- No need to shift elements.
- Time Complexity:
 - From Beginning: $O(1)$
 - From End: $O(n)$
 - At Given Position: $O(n)$
- Space Complexity: $O(1)$
- Handle special cases: empty list and single node list.

SUMMARY TABLE

Operation	Description	Time Complexity	Space Complexity
Traversing	Visit each node from HEAD to NULL	$O(n)$	$O(1)$
Insertion	Add new node	$O(1)$ to $O(n)$	$O(1)$
Deletion	Remove a node	$O(1)$ to $O(n)$	$O(1)$

IMPORTANT NOTES

- Linked list size is dynamic.
- Memory is allocated as per need.
- Efficient insertions and deletions.
- Extra memory is needed for links.
- No random access (sequential access only).

KEY TAKEAWAY



Linked list operations are efficient for insertions and deletions, but access (search/traversal) is slower compared to arrays.



WHAT IS A TREE?

A tree is a non-linear hierarchical data structure consisting of nodes connected by edges.

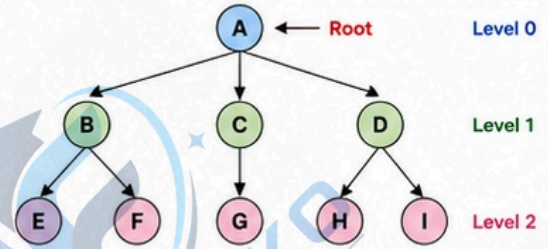
It has a special node called **Root**, and the remaining nodes are partitioned into disjoint subtrees of the root.

TREE

Definition and Terminologies, Memory Representations

- ✓ A tree with 'n' nodes has exactly (n - 1) edges.
- ✓ There is exactly one path between any two nodes.
- ✓ A tree does not contain any cycle.

EXAMPLE OF TREE



1. DEFINITION AND TERMINOLOGIES

	Root	The topmost node of the tree. It has no parent.	
	Parent	A node that has one or more children.	
	Child	A node that is directly connected below another node.	
	Siblings	Nodes that have the same parent.	
	Leaf Node / External Node	A node that does not have any children.	
	Internal Node / Non-leaf Node	A node that has at least one child.	

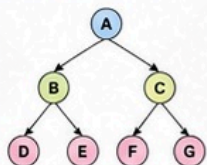
	Level	Level of a node is the number of edges from the root to that node. (Root is at level 0)	
	Height of Node	The height of a node is the number of edges on the longest path from that node to a leaf node.	
	Height of Tree	The height of a tree is the height of the root node (i.e., longest path from root to a leaf).	Height of Tree = 2
	Degree of Node	The degree of a node is the number of children it has.	Degree(C) = 1 Degree(B) = 2
	Degree of Tree	The degree of a tree is the maximum degree of any node in the tree.	Degree of Tree = 2 (because B has maximum 2 children)
	Subtree	A subtree is a tree formed by a node and all its descendants.	

2. MEMORY REPRESENTATIONS OF TREE

2.1 SEQUENTIAL (ARRAY) REPRESENTATION

- Used mainly for Binary Trees (especially Complete Binary Trees).
- Nodes are stored level by level in an array.
- If a node is at index i (0-based indexing):
 - Left child $\rightarrow 2i + 1$
 - Right child $\rightarrow 2i + 2$
 - Parent $\rightarrow \lfloor (i - 1) / 2 \rfloor$

Example Tree



Array Representation (0-based index)

Index	0	1	2	3	4	5	6
Value	A	B	C	D	E	F	G

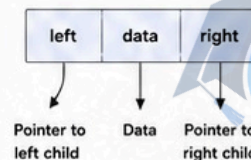
Note:

- Works efficiently when tree is complete or nearly complete.
- Not suitable for skewed (degenerate) trees as it wastes space.

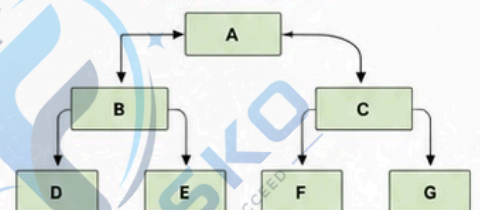
2.2 LINKED REPRESENTATION

- Each node is represented using a structure.
- Every node contains data and pointers to its children.
- For a binary tree, each node has two pointers: left and right.
- For a general tree, a node may contain multiple child pointers (using array or list).

Binary Tree Node Structure



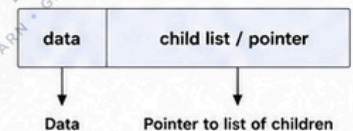
Example (Binary Tree)



Note:

- Efficient for skewed trees.
- Extra memory used for storing pointers.
- Easily supports dynamic size.

General Tree Node Structure (using list)



WHAT IS A TREE?

- A tree is a hierarchical data structure.
- It consists of nodes connected by edges.
- One special node is called **Root**.

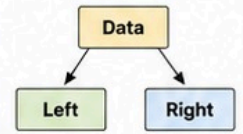
TYPES OF TREES : BINARY TREES

A Binary Tree is a tree data structure in which each node has at most two children, referred to as the **Left child** and **Right child**.

KEY PROPERTIES OF BINARY TREE

- Each node has at most 2 children.
- The left child is considered before the right child.
- The order of children matters (Left & Right).
- Binary Tree with 'n' nodes has at most (n - 1) edges.

BINARY TREE NODE



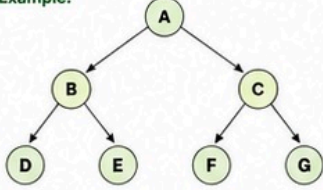
Left → Left Subtree
Right → Right Subtree

TYPES OF BINARY TREES

1. FULL (STRICT) BINARY TREE

Every node has either 0 or 2 children. No node has only one child.

Example:



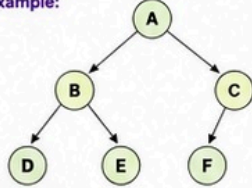
Properties:

- Every internal node has exactly 2 children.
- If the number of internal nodes = i, then number of leaves = i + 1.
- Max nodes at level i (root at level 0) = 2^i

2. COMPLETE BINARY TREE

All levels are completely filled except possibly the last level, which is filled from left to right.

Example:



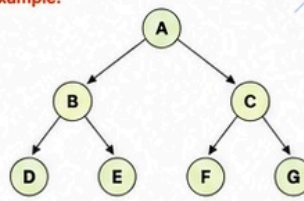
Properties:

- All levels except last are full.
- Last level nodes are as left as possible.
- For n nodes, height is minimum and $\leq \lceil \log_2 n \rceil$.

3. PERFECT BINARY TREE

All internal nodes have 2 children and all leaves are at the same level.

Example:



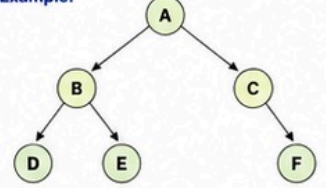
Properties:

- All levels are completely filled.
- Number of nodes = $2^{h+1} - 1$ (where h = height of tree, root at level 0)
- Number of leaves = 2^h

4. BALANCED BINARY TREE

For every node, the height difference between left and right subtree is at most 1.

Example:



Properties:

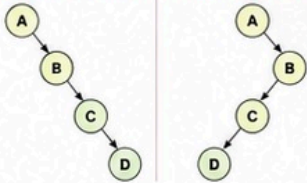
- $|\text{Height}(\text{Left Subtree}) - \text{Height}(\text{Right Subtree})| \leq 1$ for every node.
- Used in Self-balancing Trees (AVL, Red-Black, etc.).

5. DEGENERATE (SKEWED) BINARY TREE

Every node has only one child.

Left Skewed Tree

Right Skewed Tree



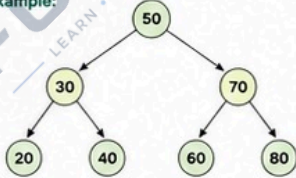
Properties:

- Each node has only one child.
- Number of nodes = height + 1.
- Behaves like a linked list.

6. BINARY SEARCH TREE (BST)

For every node:
Left subtree contains values < node data.
Right subtree contains values > node data.

Example:



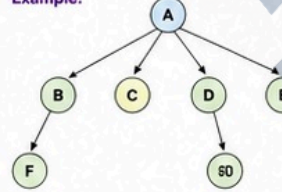
Properties:

- Left < Root < Right.
- Inorder traversal gives sorted order.
- Used in Searching, Sorting, Range queries.

7. EXTENDED / 2-ARY TREE

A generalization where a node can have more than 2 children.

Example:



Properties:

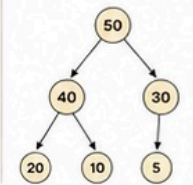
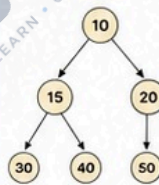
- Node can have more than 2 children.
- Not a binary tree in strict sense.
- Used in file systems, organization charts.

8. HEAP (MIN-HEAP / MAX-HEAP)

A Complete Binary Tree that satisfies Heap property.

Min-Heap Example

Max-Heap Example



Properties:

- Min-Heap: Parent $\leq$ Children.
- Max-Heap: Parent $\geq$ Children.
- Used in Priority Queues, Heap Sort.

COMPARISON OF BINARY TREE TYPES

Tree Type	Max Children per Node	All Levels Full?	Leaves at Same Level?	Height (h) for n nodes	Main Use / Feature
Full (Strict)	2	No	No	$\leq n - 1$	No node with only one child
Complete	2	Except Last	No	$\leq \lceil \log_2 n \rceil$	Efficient storage, used in Heaps
Perfect	2	Yes	Yes	$\log_2(n+1) - 1$	Ideal tree, all levels full
Balanced	2	No	No	$\leq \lceil \log_2 n \rceil$	Keeps tree height small
Degenerate (Skewed)	1	No	No	$= n - 1$	Worst case, like linked list
BST	2	No	No	Depends	Searching, Sorting operations
Extended (2-ary)	> 2	No	No	Depends	Multi-way representation
Heap	2	Complete	Not Necessarily	$\leq \lceil \log_2 n \rceil$	Priority Queue, Heap Sort

KEY TAKEAWAYS

- Binary Tree is the foundation of many advanced data structures.
- Full, Complete and Perfect trees are special forms of Binary Tree.
- Balanced trees keep operations efficient.
- BSTs maintain sorted data for fast search operations.
- Heaps are complete binary trees with special ordering.



NOTE:

All the above trees are types of Binary Trees either by structure (shape) or by property (ordering).

WHAT IS A TREE?

A tree is a non-linear data structure in which nodes are connected by edges.

One special node is called **Root** and the remaining nodes are partitioned into disjoint subtrees.

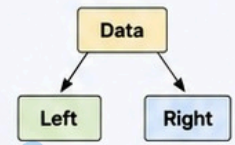
COMPLETE BINARY TREES

Definition, Properties, Examples, Representation and Uses

DEFINITION

A Complete Binary Tree is a binary tree in which all the levels are completely filled except possibly the last level, and all nodes in the last level are as far left as possible.

BINARY TREE NODE

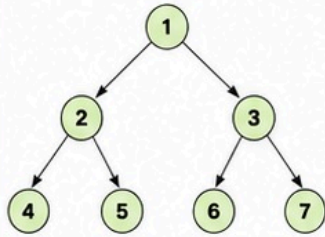


Left → Left Subtree

Right → Right Subtree

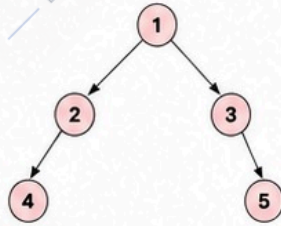
EXAMPLES

Complete Binary Tree (Valid)



- ✓ All levels are full except the last level.
- ✓ Last level nodes are as far left as possible.

Not a Complete Binary Tree (Invalid)



- ✗ Last level is not filled from the left.
- ✗ Node 5 is present but left position (under 2) is empty while a right position exists.

PROPERTIES

- 1 All levels are completely filled except possibly the last level.
- 2 All nodes of the last level are placed as far left as possible.
- 3 If a complete binary tree has 'n' nodes, the number of nodes in the last level can be from 1 to 2^h (where h = height of the tree, root at level 0).
- 4 A complete binary tree with h levels has at least 2^h nodes and at most $2^{h+1} - 1$ nodes.
- 5 Efficient for array representation and commonly used in Heap data structure.

NOTE

Every Perfect Binary Tree is a Complete Binary Tree, but every Complete Binary Tree is not necessarily Perfect.

ARRAY (SEQUENTIAL) REPRESENTATION

- In a complete binary tree, nodes can be stored in an array without wasting space.
- Nodes are stored level by level from left to right.

Array Representation (0-based Indexing)

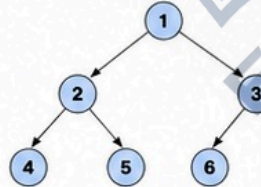
Index	0	1	2	3	4	5	6
Value	A	B	C	D	E	F	G

Index Relations (0-based indexing)

- Left child of node at index i → $2i + 1$
- Right child of node at index i → $2i + 2$
- Parent of node at index i → $(i - 1) / 2$
- Root is stored at index 0

REPRESENTATION EXAMPLE

- Consider a complete binary tree with 6 nodes.
- Height = 2 (levels 0, 1, 2)
- Last level is filled from the left.



Array (0-based indexing)

Index	0	1	2	3	4	5
Value	1	2	3	4	5	6

Why Use Array?

- ✓ No pointers needed.
- ✓ Fast access to parent/children.
- ✓ Less memory overhead.
- ✓ Ideal for heaps and priority queues.

APPLICATIONS



Heaps

Complete Binary Trees are used to implement Min-Heap and Max-Heap.



Priority Queues

Efficient insertion and deletion of elements.



Array Based Implementation

Enables efficient storage and memory utilization.



Used in Algorithms

Used in various algorithms like Heap Sort, Tree Traversals, etc.

COMPARISON

Feature	Complete Binary Tree	Perfect Binary Tree
All levels full?	Except possibly last	Yes
Last level nodes	As far left as possible	Completely filled
Number of nodes at level i (0-based)	At most 2^i	Exactly 2^i
Total nodes in h levels	2^h to $2^{h+1} - 1$	$2^{h+1} - 1$
Example		

KEY TAKEAWAYS

- ★ All levels are full except possibly the last level.
- ★ Last level nodes are as far left as possible.
- ★ Efficient array representation.
- ★ Used widely in heaps and priority queues.
- ★ Number of nodes in h levels is between 2^h and $2^{h+1} - 1$.



Quick Fact

If a complete binary tree has n nodes, then height $h = \lceil \log_2 n \rceil$



BST OPERATIONS

INSERTION & DELETION



Maintain BST property by comparing keys and moving left or right

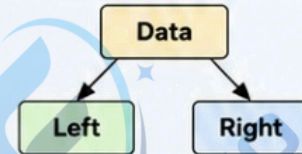
BST PROPERTY

- ✓ All keys in left subtree < node key
- ✓ All keys in right subtree > node key

WHAT IS BST?

A Binary Search Tree (BST) is a binary tree in which, for every node, all keys in the left subtree are smaller and all keys in the right subtree are greater.

NODE STRUCTURE



1. INSERTION IN BST

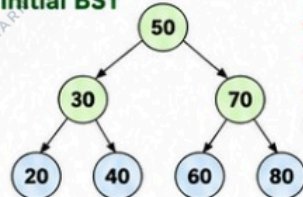
Insert a new key by finding the correct position using comparisons.

ALGORITHM

- 1 If the tree is empty, create the new node and make it root.
- 2 Compare the key with current node.
 - If key < node key → go to left subtree.
 - If key > node key → go to right subtree.
- 3 Repeat until an empty position is found.
- 4 Insert the new node at that position.

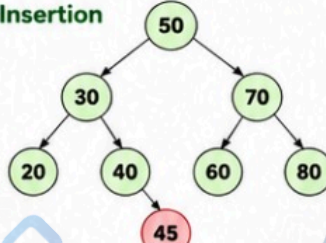
EXAMPLE: Insert 45

Initial BST



45 < 50 → go left
45 > 30 → go right
45 > 40 → go right (empty) → insert here

BST after Insertion



TRACE TABLE

Step	Current Node	Comparison	Move
1	50	45 < 50	Left
2	30	45 > 30	Right
3	40	45 > 40	Right
4	NULL	Insert 45	Stop

TIME COMPLEXITY

- Best Case : $O(1)$
(tree is empty)
- Average Case : $O(\log n)$
(balanced BST)
- Worst Case : $O(n)$
(skewed BST)

2. DELETION IN BST

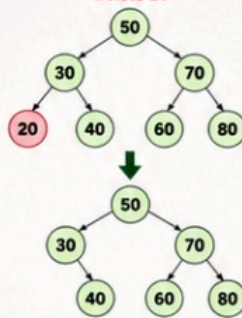
Delete a node and maintain BST property.

ALGORITHM

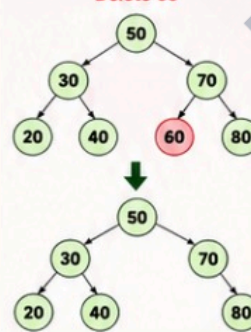
- 1 Find the node to be deleted using BST search.
- 2 There are three cases:
 - Case 1 : No child
 - Case 2 : One child
 - Case 3 : Two children
- 3 Perform the appropriate case and maintain BST property.

THREE CASES OF DELETION

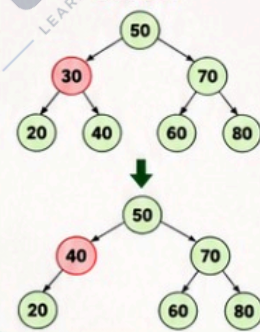
Case 1 : No Child (Leaf Node) Delete 20



Case 2 : One Child Delete 60



Case 3 : Two Children Delete 30



How Case 3 Works?

Find in-order successor (smallest in right subtree) or in-order predecessor (largest in left subtree), replace the node, then delete that successor/predecessor.

Example (Case 3 using Inorder Successor)

1. Inorder successor of 30 is 40.
2. Replace 30 with 40.
3. Delete the successor node 40 from its original position.

Inorder Sequence

Before : 20, 30, 40, 50, 60, 70, 80

After : 20, 40, 50, 60, 70, 80

(Sorted order maintained)

TIME COMPLEXITY (Deletion)

- Best Case : $O(1)$
- Average Case : $O(\log n)$
- Worst Case : $O(n)$
(Same as search)



NOTE

After every insertion or deletion, the BST property must remain true for the tree.

(Left < Node < Right)



KEY TAKEAWAYS

Insertion is done at a leaf position after comparisons.



Deletion has 3 cases: No child, One child, Two children.



BST operations are efficient for balanced trees ($O(\log n)$).

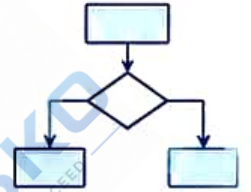


Skewed BST behaves like a linked list ($O(n)$).

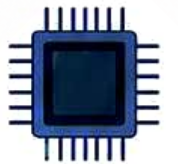


Better Data Structures → Better Programs → A Better You!

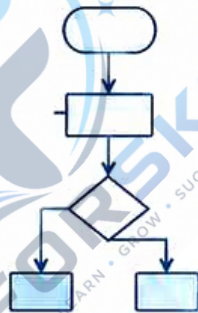
1010101010
0101010101
1010101010
0101010101



10110
01001
10101



Unit - IV





SEARCHING

DEFINITION AND CONCEPT



1. DEFINITION

Searching is the process of finding the location (position) of a specific element (key) in a collection of elements (data set). If the element is found, the algorithm returns its position, otherwise it indicates that the element is not present.

In Simple Words

Searching means looking for a particular item in a collection to know whether it exists or not and where it is located.

Visual Example

Index	0	1	2	3	4	5	6
Data	12	25	37	42	55	68	90

Search Key = 42

Key 42 is present at index 3 (position 4th).

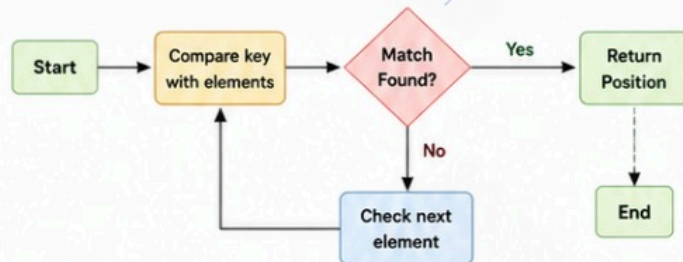
Search Key = 100

Key 100 is not present in the list.

2. CONCEPT

- ◆ We have a collection of elements stored in an array or other data structure.
- ◆ We compare the search key with the elements according to a strategy.
- ◆ If a match is found → return its position.
- ◆ If all elements are checked and no match is found → return NOT FOUND.

General Searching Process



3. TYPES OF SEARCHING

(A) Linear Search (Sequential Search)

Elements are checked one by one from the beginning until the key is found or the list ends.

Example:

Index	0	1	2	3	4	5	6
Data	12	25	37	42	55	68	90

Search Key = 37

Comparisons:

12 × → 25 × → 37 ✓ (Found at index 2)

Works on both sorted and unsorted data.

(B) Binary Search

Used on **sorted data**. The list is divided into halves. Key is compared with middle element.

Example (Sorted Data):

Index	0	1	2	3	4	5	6
Data	12	25	37	42	55	68	90

Search Key = 42

Steps:

1. Middle element = 42 (index 3) → Match Found ✓

Very efficient for large sorted lists.

4. IMPORTANT TERMS

	Key	The element which is to be searched.
	Position / Index	Location of the key in the data structure.
	Success	Key is found in the given collection.
	Failure	Key is not found in the given collection.
	Efficiency	Measured by the number of comparisons and time taken.

5. EXAMPLE ILLUSTRATION

Array:	15	28	36	47	59	63	75
--------	----	----	----	----	----	----	----

Linear Search

Search Key = 59

15 ×
28 ×
36 ×
47 ×
59 ✓ (Found at index 4)

Binary Search (Sorted)

Search Key = 59

1. Mid=47 → 59>47 → Right Half
2. Mid=63 → 59<63 → Left Half
3. Mid=59 → Match Found ✓
(Found at index 4)

6. TIME COMPLEXITY COMPARISON

Algorithm	Best Case	Average Case	Worst Case	Space Complexity	Applicable To
Linear Search	O(1)	O(n)	O(n)	O(1)	Sorted & Unsorted
Binary Search	O(1)	O(log n)	O(log n)	O(1)	Sorted Only

7. KEY TAKEAWAYS



Searching helps to quickly locate a desired element.



Linear Search is simple but slower for large data.



Binary Search is very fast but requires sorted data.



Choosing the right search method improves the efficiency of programs.



SEARCHING TECHNIQUES

LINEAR SEARCH



Linear Search (Sequential Search) is the simplest searching technique in which each element of the list is **checked one by one** from the beginning until the desired element (key) is found or the list ends.

1. DEFINITION

Linear Search is a searching technique that compares the search key with each element of the list sequentially from the first element to the last element. If the key matches an element, its position (index) is returned, otherwise the search is unsuccessful.

In Simple Words

We start from the first element and check every element one by one until we find the key or reach the end of the list.

2. CONCEPT

- ◆ Compare the search key with the first element.
- ◆ If match is found → return the index.
- ◆ If not matched → move to the next element.
- ◆ Repeat the process until key is found or we reach the end of the list.
- ◆ If key is not found → return NOT FOUND.

3. HOW LINEAR SEARCH WORKS

Index	0	1	2	3	4	5	6	7
Array	25	17	31	44	58	13	99	27

Search Key = 44

Step 1:	25	17	31	44	58	13	99	27	25 ≠ 44 → Move Next
Step 2:	25	17	31	44	58	13	99	27	17 ≠ 44 → Move Next
Step 3:	25	17	31	44	58	13	99	27	31 ≠ 44 → Move Next
Step 4:	25	17	31	44	58	13	99	27	44 = 44 → Found at index 3

4. ALGORITHM

- 1 Start
- 2 Read array A[0...n-1] and search key K
- 3 Set i = 0
- 4 While i < n
 - If A[i] == K
→ Return i (index of key)
 - Else
→ i = i + 1
- 5 If loop ends and key not found
→ Return NOT FOUND
- 6 Stop

5. TRACE TABLE EXAMPLE

Array: A = [25, 17, 31, 44, 58, 13, 99, 27] Key = 44

Step	i (Index)	A[i]	Comparison A[i] == Key	Action	Result
1	0	25	25 == 44 (False)	Move to next	Continue
2	1	17	17 == 44 (False)	Move to next	Continue
3	2	31	31 == 44 (False)	Move to next	Continue
4	3	44	44 == 44 (True)	Key Found	Return index 3

(Search Successful at index 3)

6. PSEUDO CODE

```
LinearSearch(A, n, K)
1. for i = 0 to n-1
2.   if A[i] == K
3.     return i
4. return NOT FOUND
End LinearSearch
```

7. C / C++ CODE

```
int linearSearch(int A[], int n, int key)
{
    for (int i = 0; i < n; i++)
    {
        if (A[i] == key)
            return i; // key found
    }
    return -1; // key not found
}
```

Example Run:

A = [25, 17, 31, 44, 58, 13, 99, 27]

Key = 44

Output: Key found at index 3

Key = 100

Output: Key not found (-1)

8. TIME COMPLEXITY

Best Case : O(1)
(Key is at first position)
Average Case : O(n)
Worst Case : O(n)
(Key is at last or not present)
Space Complexity : O(1)

9. ADVANTAGES

- ✓ Very simple to understand
- ✓ Works on both sorted and unsorted data
- ✓ No extra space required

10. DISADVANTAGES

- ✗ Slow for large data
- ✗ Not efficient for large lists
- ✗ Many comparisons may be needed

11. APPLICATIONS

- Small lists
- Unsorted data
- Searching in files
- Checking presence of data



KEY TAKEAWAY

Linear Search is easy to implement and useful for small or unsorted data, but it requires up to n comparisons in the worst case.





BINARY SEARCH

Searching Technique for Sorted Data

Binary Search is an efficient searching technique used on **sorted** (ascending or descending) data by repeatedly dividing the search interval in half.



1. DEFINITION

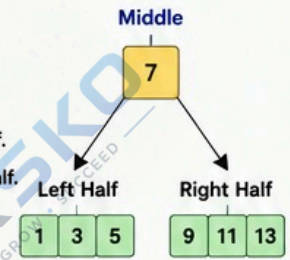
Binary Search compares the search key with the middle element of the sorted list. If they are not equal, the half in which the key cannot lie is eliminated and the search continues on the other half, again taking the middle element to compare, and so on until the key is found or the list is empty.

In Simple Words

We repeatedly divide the list into two halves, check the middle element, and continue the search in the appropriate half.

2. CONCEPT

- ✓ Data must be sorted.
- ✓ Compare the key with the middle element.
- ✓ If key = middle element → return its index.
- ✓ If key < middle element → search in left half.
- ✓ If key > middle element → search in right half.
- ✓ Repeat the process until key is found or the search interval becomes empty.
- ✓ If not found → return NOT FOUND.



3. HOW BINARY SEARCH WORKS (Example)

Sorted Array (Index)						
0	1	2	3	4	5	6
2	4	6	8	10	12	14

Step 1

low = 0, high = 6
mid = (0+6)/2 = 3

2	4	6	8	10	12	14
---	---	---	---	----	----	----

8 < 11

Search in right half

Step 2

low = 4, high = 6
mid = (4+6)/2 = 5

2	4	6	8	10	12	14
---	---	---	---	----	----	----

12 > 11

Search in left half

Step 3

low = 4, high = 4
mid = (4+4)/2 = 4

2	4	6	10	10	12	14
---	---	---	----	----	----	----

10 < 11

Search in right half

Step 4

low = 5, high = 4
(low > high)

Key not found

4. ALGORITHM

- Start
- Set low = 0, high = n - 1
- While (low ≤ high)
 - mid = (low + high) / 2
 - If A[mid] == key → return mid
 - Else if key < A[mid] → high = mid - 1
 - Else → low = mid + 1
- If loop ends → return NOT FOUND.
- Stop



Note
This works only on sorted data.

5. TRACE TABLE EXAMPLE

Array: A = [2, 4, 6, 8, 10, 12, 14] Key = 11

Step	low	high	mid	A[mid]	Comparison	Action
1	0	6	3	8	8 < 11	low = mid + 1 (4)
2	4	6	5	12	12 > 11	high = mid - 1 (4)
3	4	4	4	10	10 < 11	low = mid + 1 (5)
4	5	4	-	-	low > high	Key not found

Result: Key 11 is not present in the array.

6. PSEUDO CODE

```
BinarySearch(A, n, key)
1. low ← 0
2. high ← n - 1
3. while (low ≤ high)
    mid ← (low + high) / 2
    if (A[mid] == key)
        return mid
    else if (key < A[mid])
        high ← mid - 1
    else
        low ← mid + 1
4. return NOT FOUND
End BinarySearch
```

7. C / C++ CODE

```
int binarySearch(int A[], int n, int key)
{
    int low = 0, high = n - 1, mid;
    while (low <= high)
    {
        mid = (low + high) / 2;
        if (A[mid] == key)
            return mid;
        else if (key < A[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
    return -1; // Not Found
}
```

8. EXAMPLE RUN (Key = 11)

A = [2, 4, 6, 8, 10, 12, 14]

The steps are same as in the trace table.

Output: Key 11 is not found (- 1)

9. TIME COMPLEXITY

Best Case : O(1)
Average Case : O(log n)
Worst Case : O(log n)
Space Complexity : O(1) (iterative)
O(log n) (recursive)

10. ADVANTAGES

- ✓ Very fast for large datasets.
- ✓ Requires only O(log n) comparisons.
- ✓ Easy to implement (iterative).
- ✓ Works efficiently on sorted data.

11. DISADVANTAGES

- ✗ Data must be sorted.
- ✗ Insertion/Deletion in sorted array is costly.
- ✗ Not suitable for unsorted data.

12. APPLICATIONS

- Searching in large sorted lists.
- Used in databases and files.
- Dictionary lookups.
- Routing and network algorithms.
- Many real-world systems.



KEY TAKEAWAY

Binary Search is much faster than Linear Search and is ideal for large, sorted datasets, reducing the search space by half in each step.





INDEXED SEQUENTIAL SEARCH

A searching technique that combines the advantages of Sequential Search and Binary Search.



1. DEFINITION

Indexed Sequential Search is a searching technique for ordered data that maintains an index (a secondary list) with key values and their addresses (or positions).

It first performs a binary search on the index to find the block where the key may exist, and then performs a sequential search within that block.

In Simple Words

We first search in the index using binary search to find the appropriate block, then search sequentially in that block.

2. CONCEPT

- ◆ Data is sorted and divided into blocks.
- ◆ An index is created for the first element of each block along with its address.
- ◆ Binary search is applied on the index.
- ◆ Once the block is located, linear search is applied within that block.
- ◆ If key is found → return its position.
- ◆ If key is not found → return NOT FOUND.

Index Table (Primary Index)

Key	Address (Block No.)
10	0
30	1
50	2
70	3

Search in appropriate block sequentially

3. DATA STRUCTURE EXAMPLE

Assume Block Size = 4

Main Data (Ordered)

Index :	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Data :	10	12	15	18	30	31	35	40	50	55	60	65	70	72	78	85

Block 0 Block 1 Block 2 Block 3

Index Table

Key (First element)	10	30	50	70
Address (Block No.)	0	1	2	3

4. HOW IT WORKS (Example)

Search Key = 60

Step 1: Binary search on Index Table

Index Table Keys: [10, 30, 50, 70]

60 > 50 and 60 < 70 → Possible in Block 3 (Address = 3)

Step 2: Sequential search in Block 3

Block 3 Elements: [70, 72, 78, 85]

60 is not in this block.

Hence, Key 60 is NOT FOUND.

5. ALGORITHM

- 1 Create an index table for the ordered data (first key of each block and its address).
- 2 Perform binary search on the index table for the given key.
- 3 If exact match is found in index → return its position.
- 4 Otherwise, get the address of the block where the key may exist.
- 5 Apply sequential search within that block.
- 6 If key is found → return its position.
- 7 If not found in the block → return NOT FOUND.
- 8 If key is not in index range → return NOT FOUND.
- 9 Stop.

Note

Block size (b) is predefined.
Data must be sorted.

6. TRACE TABLE EXAMPLE (Key = 60)

Index Table: [10, 30, 50, 70]

Block Size = 4

Step	low	high	mid	Index[mid]	Comparison	Result
1	0	3	1	30	60 > 30	low = mid + 1 (2)
2	2	3	2	50	60 > 50	low = mid + 1 (3)
3	3	3	3	70	60 < 70	Stop → Block 3

Sequential Search in Block 3

Element Checked	70	72	78	85
Comparison with 60	60 < 70	60 < 72	60 < 78	60 < 85
Result	Not Found			

Key 60 is NOT FOUND.

7. PSEUDO CODE

```
IndexedSequentialSearch(A, Index, n, b, key)
1. low ← 0, high ← [n/b] - 1
   // number of blocks
2. // Binary search on index
3. while (low ≤ high)
4.     mid ← (low + high) / 2
5.     if (Index[mid].key == key) return Index[mid].addr
6.     else if (Index[mid].key < key) low ← mid + 1
7.     else high ← mid - 1
8. // Get block address
9. block ← (low - 1) // last index where key < Index[mid].key
10. start ← block * b
11. end ← min(start + b - 1, n - 1)
12. // Sequential search in the block
13. for i ← start to end
14.     if (A[i] == key) return i
15. return NOT FOUND
```

8. C CODE (Example)

```
#define MAX 100
#define BLOCK 4

typedef struct { int key; int addr; } Index;
int indexedSequentialSearch(int A[], Index idx[], int n, int b, int key)
{
    int low = 0, high = (n + b - 1) / b - 1, mid, block, start, end, i;
    // Binary search on index
    while (low <= high)
    {
        mid = (low + high) / 2;
        if (idx[mid].key == key) return idx[mid].addr;
        else if (idx[mid].key < key) low = mid + 1;
        else high = mid - 1;
    }
    block = low - 1;
    start = block * b;
    end = (start + b - 1 < n - 1) ? start + b - 1 : n - 1;
    // Sequential search in block
    for (i = start; i <= end; i++)
        if (A[i] == key) return i;
    return -1; // Not Found
}
```

9. COMPLEXITY ANALYSIS

Let n = total number of elements
b = block size

Number of blocks = $\lceil n/b \rceil$

- Binary search on index: $O(\log(n/b))$
- Sequential search in block: $O(b)$
- Total Time Complexity: $O(\log(n/b) + b)$

Best Case: $O(1)$ (key found in index)

Worst Case: $O(\log(n/b) + b)$

Space Complexity: $O(n/b)$ for index table

10. ADVANTAGES / DISADVANTAGES

Advantages

- ✓ Faster than linear search for large datasets.
- ✓ Efficient for data that is stored on secondary storage (e.g., disk).
- ✓ Requires less comparison than linear search.

Disadvantages

- ✗ Extra space needed for index table.
- ✗ Performance depends on block size.
- ✗ More complex to implement than linear search.

11. APPLICATIONS

- Searching in large ordered files stored on disks.
- Databases and library management systems.
- Used in indexing of dictionaries, phone books, and catalogs.
- Any system where data is stored in sorted blocks.



KEY TAKEAWAY

Indexed Sequential Search reduces the number of comparisons by first searching the index (using binary search) to locate the block and then performing sequential search within that block.





SORTING



DEFINITION AND CONCEPT

Sorting is the process of arranging data or elements in a particular order (**ascending** or **descending**) based on a key or value.

1. DEFINITION

Sorting is the systematic arrangement of data elements in a specific order. The order may be either ascending (smallest to largest) or descending (largest to smallest) according to the values of the elements or a specified key.



In Simple Words

Sorting means putting the items in order so that we can easily search, process, and manage the data.

2. CONCEPT

- Sorting arranges data in a meaningful sequence.
- It is used to organize data for efficient searching, processing, and presentation.
- Elements are compared and rearranged based on a sorting order.
- After sorting, data can be in:
 - Ascending Order** : Smallest to Largest
 - Descending Order** : Largest to Smallest

3. EXAMPLE

Ascending Order (Smallest to Largest)

Unsorted: 45 12 78 33 9 56 23 → Sorted: 9 12 23 33 45 56 78

Descending Order (Largest to Smallest)

Unsorted: 45 12 78 33 9 56 23 → Sorted: 78 56 45 33 23 12 9

4. IMPORTANCE OF SORTING



Improves Search Efficiency

Sorted data allows faster searching (using binary search).



Better Data Organization

Makes data easier to read, understand, and manage.



Efficient Processing

Many algorithms and operations work better on sorted data.



Useful in Real Life

Used in databases, file systems, reporting, leaderboards, scheduling, etc.

5. GENERAL SORTING PROCESS



Input Data

Take the list/array of elements.



Compare Elements

Compare elements based on a key.



Rearrange Elements

Swap or shift elements to place them in correct order.



Repeat

Repeat the process until the entire list is sorted.



Sorted Output

The list is now in ascending or descending order.

6. TYPES OF SORTING ORDER

Ascending Order

Example: 5 2 9 1 7 → 1 2 5 7 9

Descending Order

Example: 5 2 9 1 7 → 9 7 5 2 1

7. APPLICATIONS OF SORTING



Database Management



File and Record Organization



E-commerce (Product Listing)



Rankings and Leaderboards



Scheduling and Timetabling



Data Analysis and Reporting

8. CHARACTERISTICS OF A GOOD SORTING ALGORITHM

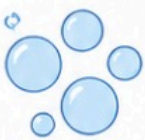
Characteristic	Description
Correctness	Always produces correctly sorted output.
Efficiency	Uses minimum time and memory.
Stability	Maintains the relative order of equal elements.
In-place	Requires less extra memory.
Scalability	Performs well for large datasets.



KEY TAKEAWAY

Sorting organizes data in a specific order to make it easier to search, process, and analyze. It is a fundamental operation in computer science with numerous real-world applications.





SORTING TECHNIQUES

BUBBLE SORT



Bubble Sort is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements and swaps them if they are in the wrong order. This process is repeated until the list is sorted.

1. DEFINITION

Bubble Sort is a comparison-based sorting algorithm. It gets its name because larger elements "bubble" toward the end of the list in each pass, just like bubbles rise to the surface in water.

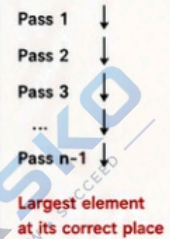
In Simple Words

Keep comparing neighboring elements and swap them if they are in the wrong order. Repeat until no more swaps are needed.

2. CONCEPT

- Compare adjacent elements.
- If the left element is greater than the right element, swap them.
- After each pass, the largest unsorted element moves to its correct position at the end.
- Reduce the range of comparison by one after each pass.
- Repeat until no swaps are needed in a pass.

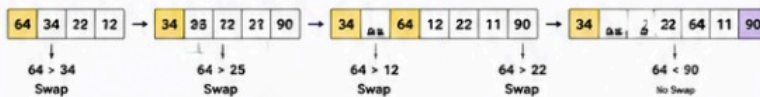
How it "Bubbles"



3. WORKING EXAMPLE (Ascending Order)

Array: 64, 34, 25, 12, 22, 11, 90 (n = 7)

Pass 1: (Largest element goes to last position)



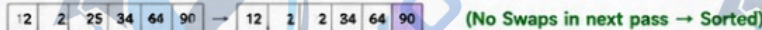
Pass 2:



Pass 3:



Pass 4:



Key

- 64: Element being compared
- 34: Element being compared with
- 64: Element after swapping
- 90: Element at correct position after each pass

4. ALGORITHM

- Start
- For $i = 0$ to $n-2$
- For $j = 0$ to $n-i-2$
- If $A[j] > A[j+1]$
Swap $A[j]$ and $A[j+1]$
- Next j
- Next i
- Stop

Note

After i th pass, the largest element in the unsorted part is placed at position $n-i-1$.

5. PSEUDO CODE

```
BubbleSort(A, n)
1. for i = 0 to n-2
2.   for j = 0 to n-i-2
3.     if A[j] > A[j+1]
4.       swap A[j] and A[j+1]
5.   next j
6. next i
End BubbleSort
```

6. C CODE

```
void bubbleSort(int A[], int n)
{
    int i, j, temp;
    for (i = 0; i < n-1; i++)
    {
        for (j = 0; j < n-i-1; j++)
        {
            if (A[j] > A[j+1])
            {
                temp = A[j];
                A[j] = A[j+1];
                A[j+1] = temp;
            }
        }
    }
}
```

7. TRACE TABLE EXAMPLE (Array: [64, 34, 25, 12, 22, 11, 90])

Pass	Comparisons & Swaps	Array After Pass	Largest at Position
Initial	-	64, 34, 25, 12, 22, 11, 90	-
Pass 1	Multiple swaps	34, 25, 12, 22, 11, 64, 90	Index 6 (90)
Pass 2	Multiple swaps	25, 12, 22, 11, 34, 64, 90	Index 5 (64)
Pass 3	Multiple swaps	12, 22, 11, 25, 34, 64, 90	Index 4 (34)
Pass 4	Multiple swaps	12, 11, 22, 25, 34, 64, 90	Index 3 (25)
Pass 5	1 swap	11, 12, 22, 25, 34, 64, 90	Index 2 (22)
Pass 6	0 swap → Stop	11, 12, 22, 25, 34, 64, 90 (Sorted)	-

8. CHARACTERISTICS

- Simple and easy to understand
- In-place sorting (no extra space)
- Stable sort (maintains relative order of equal elements)
- Adaptive (can stop early if already sorted using a flag)
- Not efficient for large datasets

9. COMPLEXITY ANALYSIS

Case	Comparisons	Swaps	Time Complexity
Best Case (Already Sorted)	$n - 1$	0	$O(n)$
Average Case	$n(n - 1) / 2$	$\sim n^2 / 4$	$O(n^2)$
Worst Case (Reversed Order)	$n(n - 1) / 2$	$n(n - 1) / 2$	$O(n^2)$

Space Complexity: $O(1)$ (In-place)

10. ADVANTAGES

- Easy to implement
- Requires only constant extra space
- Works well for small or nearly sorted lists
- Stable sorting algorithm

11. DISADVANTAGES

- Very inefficient for large datasets
- Higher number of comparisons and swaps
- Slower performance compared to advanced algorithms

KEY TAKEAWAY



Bubble Sort is a simple comparison-based algorithm where larger elements "bubble" to the end in each pass until the entire list is sorted.





SELECTION SORT

Simple Idea, Powerful Concept!



WHAT IS SELECTION SORT?

Selection Sort is a simple comparison-based sorting algorithm. It repeatedly selects the smallest (or largest) element from the unsorted part and moves it to the sorted part.

KEY IDEA

Find the minimum element from the unsorted part and place it at the beginning.

ALGORITHM

- 1 Start from the first element.
- 2 Find the minimum element in the unsorted part.
- 3 Swap the minimum element with the first element of the unsorted part.
- 4 Move the boundary of the sorted part one step forward.
- 5 Repeat steps 2–4 until the array is sorted.

PSEUDOCODE

```
for i = 0 to n-2
  min_idx = i
  for j = i+1 to n-1
    if arr[j] < arr[min_idx]
      min_idx = j
  swap(arr[i], arr[min_idx])
```

COMPLEXITY ANALYSIS

Best Case	$O(n^2)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$
Space Complexity	$O(1)$

EXAMPLE

Sort the following array in ascending order.

Initial Array :

64 25 12 22 11

STEP-BY-STEP DRY RUN

Pass 1

Find the minimum (11) in the whole array and swap it with the first element.

64 25 12 22 11 → Swap 64 and 11

11 25 12 22 64
11 is in correct position

Pass 2

Find the minimum (12) in the remaining unsorted part and swap it with the first element of unsorted part.

11 25 12 22 64 → Swap 25 and 12

11 12 25 22 64
12 is in correct position

Pass 3

Find the minimum (22) in the remaining unsorted part and swap it.

11 12 25 22 64 → Swap 25 and 22

11 12 22 25 64
22 is in correct position

Pass 4

Find the minimum (25) in the remaining unsorted part and swap it.

11 12 22 25 64 → Swap 25 and 25 (No change)

11 12 22 25 64
25 is in correct position



ARRAY IS SORTED!

11 12 22 25 64

CHARACTERISTICS

- ★ In-place sorting (no extra memory required)
- ★ Not stable (relative order of equal elements may change)
- ★ Simple to understand and implement
- ★ Performs well on small datasets



PROS & CONS

PROS

- ✓ Easy to implement
- ✓ In-place ($O(1)$ space)
- ✓ Works well for small arrays

CONS

- ✗ Inefficient for large datasets ($O(n^2)$)
- ✗ Not stable
- ✗ Many comparisons even if array is sorted



INSERTION SORT



★ Builds the sorted list one item at a time! ★

WHAT IS INSERTION SORT?

Insertion Sort is a simple sorting algorithm that builds the final sorted array one item at a time. It takes each element from the unsorted part and inserts it into its correct position in the sorted part.

KEY IDEA

Take one element (key) from the unsorted part and insert it into the correct position in the sorted part.



ALGORITHM

- 1 Start from the second element (index 1).
- 2 Take the current element (key).
- 3 Compare it with elements in the sorted part (left side).
- 4 Shift all greater elements one position to the right.
- 5 Insert the key in its correct position.
- 6 Repeat steps 2–5 until the array is sorted.

PSEUDOCODE

```
for i = 1 to n-1
  key = arr[i]
  j = i - 1
  while j >= 0 and arr[j] > key
    arr[j+1] = arr[j] // shift right
    j = j - 1
  arr[j+1] = key // insert key
```

COMPLEXITY ANALYSIS

Best Case (Already Sorted)	$O(n)$
Average Case	$O(n^2)$
Worst Case (Reverse Sorted)	$O(n^2)$
Space Complexity	$O(1)$

EXAMPLE

Sort the following array in ascending order.

Initial Array :

12 11 13 5 6

Sorted Part
Key (Current)

STEP-BY-STEP DRY RUN

Pass 1
(i = 1)

- Key = 11
- Compare with 12 (greater), shift 12 right.
- Insert 11 in its correct position.



Key = 11

11 is inserted at the beginning.

Pass 2
(i = 2)

- Key = 13
- Compare with 12 (smaller).
- No shifting needed.
- Insert 13 after 12.



13 is already in correct position.

Pass 3
(i = 3)

- Key = 5
- Compare with 13, 12, 11 (all greater).
- Shift all to the right.
- Insert 5 at the beginning.



5 is inserted at the beginning.

Pass 4
(i = 4)

- Key = 6
- Compare with 13, 12, 11, 5.
- Shift 13, 12, 11 to the right.
- Insert 6 after 5.



6 is inserted in its correct position.



ARRAY IS SORTED!

5 6 11 12 13

CHARACTERISTICS

- ★ In-place sorting algorithm
- ★ Stable (maintains relative order of equal elements)
- ★ Efficient for small datasets
- ★ Online algorithm (works as data comes in)
- ★ Adaptive (good for nearly sorted data)



PROS & CONS

PROS

- ✓ Simple to understand and implement
- ✓ In-place ($O(1)$ space)
- ✓ Stable
- ✓ Works well for small or nearly sorted arrays

CONS

- ✗ Inefficient for large datasets ($O(n^2)$)
- ✗ More comparisons and shifts in worst case



TIP: Insertion Sort is efficient for small arrays or when the input array is nearly sorted.



RADIX SORT

Sorts numbers digit by digit!

WHAT IS RADIX SORT?

Radix Sort is a non-comparison based sorting algorithm. It sorts numbers digit by digit using a stable sorting technique (like Counting Sort) as a subroutine.

KEY IDEA

Sort by the least significant digit (LSD) first, then next digit, and so on until the most significant digit (MSD).



ALGORITHM

- Find the maximum number to know the number of digits.
- For each digit position starting from LSD to MSD:
 - Perform Counting Sort based on that digit.
 - Maintain stability.
- After the last digit is processed, the array is sorted.

EXAMPLE

Sort the following numbers:

170 45 75 90 802 24 2 66

Pass 1: Sort by Ones place (LSD)

Ones : 0 5 5 0 2 4 2
↓ ↓ ↓ ↓ ↓ ↓ ↓
After sorting: 90 170 802 2 24 45 75 66
Sorted by ones digit

Pass 2: Sort by Tens place

Tens : 9 7 0 0 2 4 7 6
↓ ↓ ↓ ↓ ↓ ↓ ↓
After sorting: 802 2 24 45 66 170 75 90
Sorted by tens digit

Pass 3: Sort by Hundreds place (MSD)

Hundreds : 8 0 0 0 0 1 0 0
↓ ↓ ↓ ↓ ↓ ↓ ↓
After sorting: 2 24 45 66 75 90 170 802
Sorted by hundreds digit

✓ ARRAY IS SORTED!

2 24 45 66 75 90 170 802

COMPLEXITY

Time Complexity:
 $O(d \times (n + k))$
Where n = numbers,
 d = number of digits,
 k = base (usually 10)
Space Complexity: $O(n + k)$

PROS

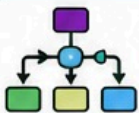
- ✓ Very efficient for large numbers.
- ✓ Linear time complexity for fixed number of digits.
- ✓ Stable sorting algorithm.

CONS

- ✗ Not suitable for comparison-based data (e.g., strings without conversion).
- ✗ Requires extra space.
- ✗ Performance depends on the range of digits.



Best for: Sorting large integers or strings with fixed length.



MERGE SORT

Divide, Conquer and Combine!

WHAT IS MERGE SORT?

Merge Sort is a comparison-based sorting algorithm that uses the Divide and Conquer approach.

ALGORITHM

- Divide the array into two halves.
- Recursively sort both halves.
- Merge the two sorted halves into a single sorted array.

COMPLEXITY

Time Complexity:
Best / Average / Worst: $O(n \log n)$
Space Complexity: $O(n)$
Stable: Yes

EXAMPLE

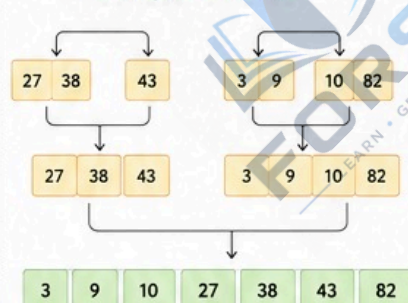
Sort the following array:

38 27 43 3 9 82 10

DIVIDE



CONQUER & MERGE



Best for: Large datasets, linked lists, external sorting.

PROS

- ✓ Guaranteed $O(n \log n)$ time.
- ✓ Stable sorting.
- ✓ Efficient for large datasets.

CONS

- ✗ Requires extra space $O(n)$.
- ✗ Not an in-place sorting algorithm.



QUICK SORT

Pick a Pivot and Partition!

WHAT IS QUICK SORT?

Quick Sort is a comparison-based sorting algorithm. It picks an element as pivot and partitions the array into two parts: elements less than pivot and greater than pivot.

ALGORITHM

- Choose a pivot element.
- Partition the array:
 - Elements $<$ pivot to the left
 - Elements $>$ pivot to the right
- Recursively apply steps 1–2 to the left and right subarrays.
- Base case: If subarray has 0 or 1 element, it is already sorted.

EXAMPLE

Sort the following array:

10 7 8 9 1 5

Step 1: Choose pivot = 10 (last element)

10 7 8 9 1 5 → Partition → 7 8 9 1 5 10
Pivot in correct place

Step 2: Sort left part [7, 8, 9, 1, 5], pivot = 5

7 8 9 1 5 → Partition → 1 5 9 7 8

Step 3: Sort left part [1], pivot = 1

1 → Partition → 1

Step 4: Sort right part [9, 7, 8], pivot = 8

9 7 8 → Partition → 7 8 9
Pivot in correct place

Final Sorted Array: 1 5 7 8 9 10

COMPLEXITY

Time Complexity:
Best Case: $O(n \log n)$
Average Case: $O(n \log n)$
Worst Case: $O(n^2)$
Space Complexity:
 $O(\log n)$ average (recursion stack)

PROS

- ✓ In-place sorting (less extra space).
- ✓ Usually faster in practice.
- ✓ Efficient for large datasets.

CONS

- ✗ Worst case $O(n^2)$ when pivot is worst (e.g., already sorted).
- ✗ Not stable.
- ✗ Performance depends on pivot choice.



Best for: General purpose sorting, in-memory sorting.



Tip: Choosing a good pivot (like random or median-of-three) improves performance.